

Grado en Ingeniería Informática

Tecnologías de Desarrollo de Software

Tema 1.

Introducción al Desarrollo de Software



Departamento
Informática y
Sistemas

Índice de Contenidos

- **Ingeniería del Software**
 - Definición y Evolución
 - Importancia en la actualidad
- Ciclo de vida del Desarrollo del Software (SDLC)
 - Etapas y Modelos del SDLC
- Requisitos Software
 - Definición y tipos
 - Casos de uso e Historias de Usuario
- Modelado del Software con UML
 - Modelado de la estructura: Diagramas de clase
 - Modelado de las interacciones: Diagramas de Secuencia y Comunicación



The Top 100
Software
Companies
of 2021

Top 100 Software

awardee list is comprised of a wide range of companies from the most well-known such as Microsoft, Adobe, and Salesforce to the relatively newer but rapidly growing - Qualtrics, Atlassian, and Asana. A good number of awardees may be new names to some but that should be no

surprise given software has always been an industry of startups that seemingly came out of nowhere to create and dominate a new space.

Software has become the backbone of our economy. From large enterprises to small businesses, most all rely on software whether for accounting, marketing, sales, supply chain, or a myriad of other functions. Software has become the dominant industry of our time and as such, we place a significance on highlighting the best companies leading the industry forward.

Consenso mundial: “Software es la industria dominante en el siglo XXI”

Ingeniería del Software: Definición

Disciplina centrada en el desarrollo, operación y mantenimiento de software de manera sistemática, disciplinada y cuantificable, aplicando principios de ingeniería, así como métodos y herramientas, con el objetivo de producir software de calidad que sea eficiente, fiable, seguro, mantenible y que satisfaga los requisitos especificados por los usuarios.

Ciclo de Vida del Desarrollo del Software (SDLC)

Proceso sistemático y organizado en etapas utilizado para desarrollar software, destinado a asegurar la calidad y productividad (reducir tiempo y esfuerzo para reducir costes).

Etapas: Captura de Requisitos, Análisis y Diseño del Sistema, Implementación, Testing, Despliegue, y Mantenimiento/Operación

Modelos (Procesos): Cascada, Incremental, Espiral, RAD, Ágil

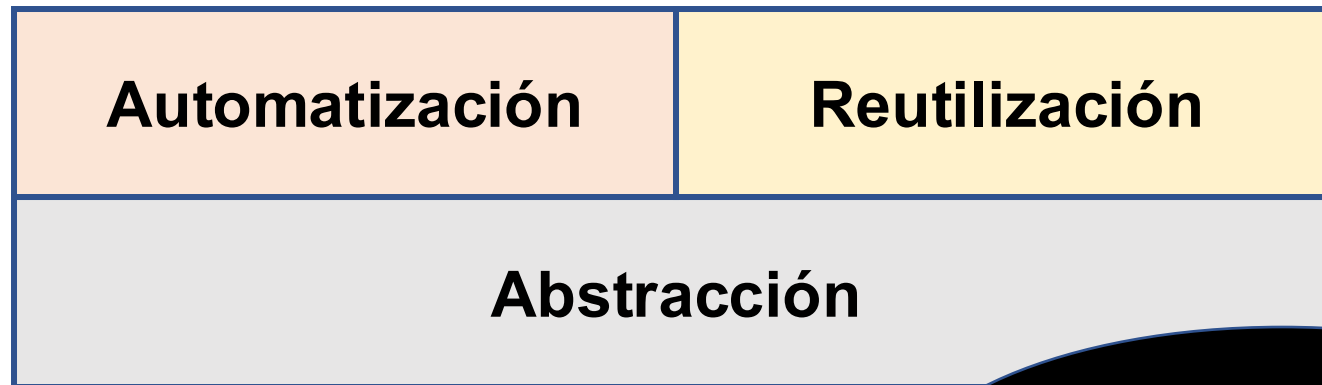
Principales Áreas de la Ingeniería del Software

- Ingeniería de Requisitos
- Procesos Software
- Diseño y Construcción de Software
- Calidad del Software
- Pruebas del Software
- Evolución del Software
- Interfaces Persona-Ordenador
- Cuestiones Éticas y Profesionales

Objetivo de la Ingeniería del Software

*Producir software con un **enfoque industrial** destinado a conseguir una alta productividad.*

*Capacidad de producir software de **alta calidad a bajo coste***



Los avances se producen en estos 3 aspectos

Why Software Is Eating The World

Software is eating the world.

- Internet de banda ancha.
- Servicios cloud computing a bajo coste.
- Nuevas grandes empresas exitosas son grandes productoras de software: Amazon, Google, Pixar, Skype, Spotify,..

...com bubble, a dozen or so new Internet companies like Facebook and Zynga, due to their rapidly growing private market valuations, and even the heyday of Webvan and Pets.com still fresh in the investor psyche, is the bubble?"

...de of the case. (I am co-founder and general partner of venture capital fund in Facebook, Groupon, Skype, Twitter, Zynga, and Foursquare, and LinkedIn.) We believe that many of the prominent new Internet companies are highly defensible businesses.

Today's stock market actually hates technology, as shown by all-time low price/earnings ratios for major public technology companies. Apple, for example, has a P/E ratio of around 15.2 — about the same as the broader stock market, despite Apple's immense profitability and dominant market position (Apple in the last couple weeks became the biggest company in America, judged by market capitalization, surpassing Exxon Mobil). And, perhaps most telling, you can't have a bubble when people are constantly screaming "Bubble!"

But too much of the debate is still around financial valuation, as opposed to the underlying intrinsic value of the best of Silicon Valley's new companies. My own theory is that we are in the middle of a dramatic and broad technological and economic shift in which software companies are poised to take over large swathes of the economy.

More and more major businesses and industries are being run on software and delivered as online services — from movies to agriculture to national defense. Many of the winners are Silicon Valley-style entrepreneurial technology companies that are invading and overturning established industry structures. Over the next 10 years, I expect many more industries to be disrupted by software, with new world-beating Silicon Valley companies doing the disruption in more cases than not.

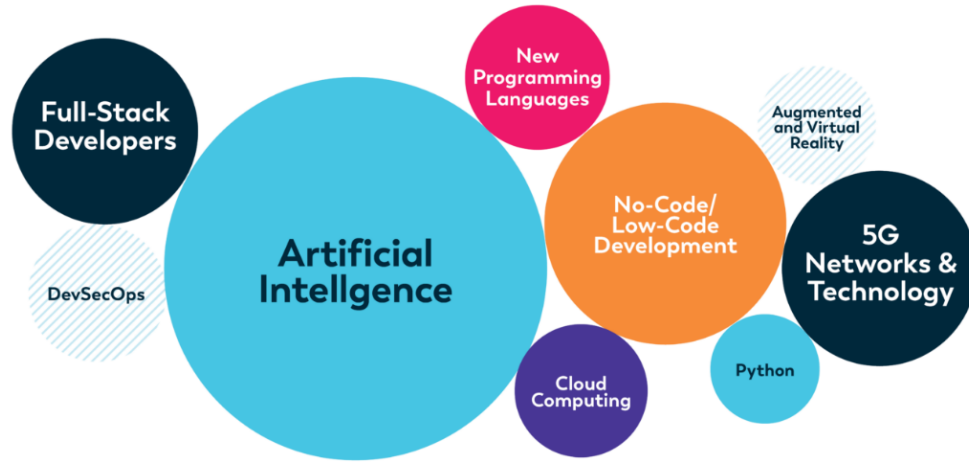
A photograph showing several people working on laptops in a modern office or co-working space. In the foreground, a man with dark hair is seen from behind, wearing large black headphones and typing on a laptop. To his left, a woman with long dark hair is also working on a laptop. In the background, another person is visible, wearing a grey t-shirt with the words 'BEAR' and 'BOOTS' printed on it. The scene is lit with warm, natural light, and there are some plants and water bottles on the desks.

TENDENCIAS · 28/03/2024

La demanda de software está superando rápidamente la oferta de talento calificado

Si bien se espera que el software continúe con un crecimiento exponencial, la

2023 Software Development Trends



Latest Software Development Trends in 2023

#1 ChatGPT Adoption in Programming

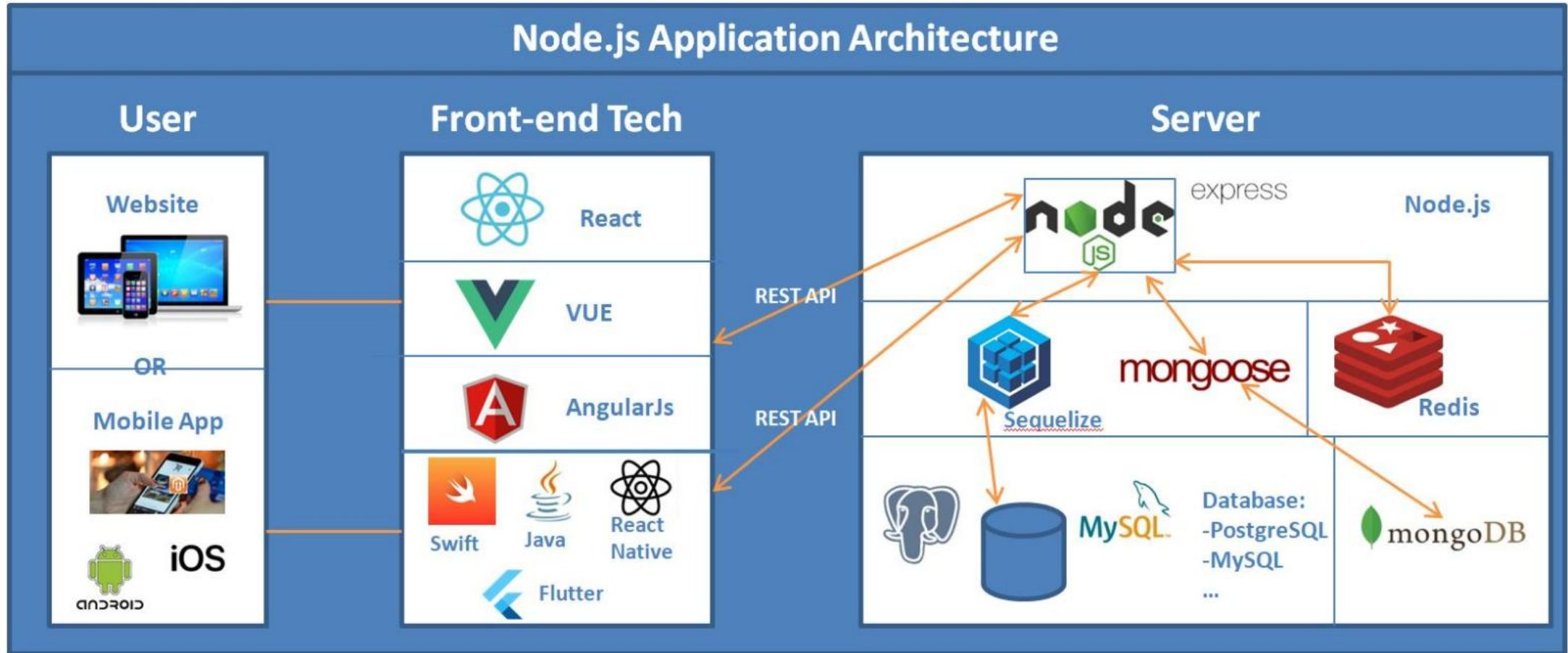
Being named the fastest-growing app of all time, Chat GPT has made a big splash in the tech community and has become a popular software industry trend. Just in five days, the app acquired its first million users while it took other apps with household names months and even years.

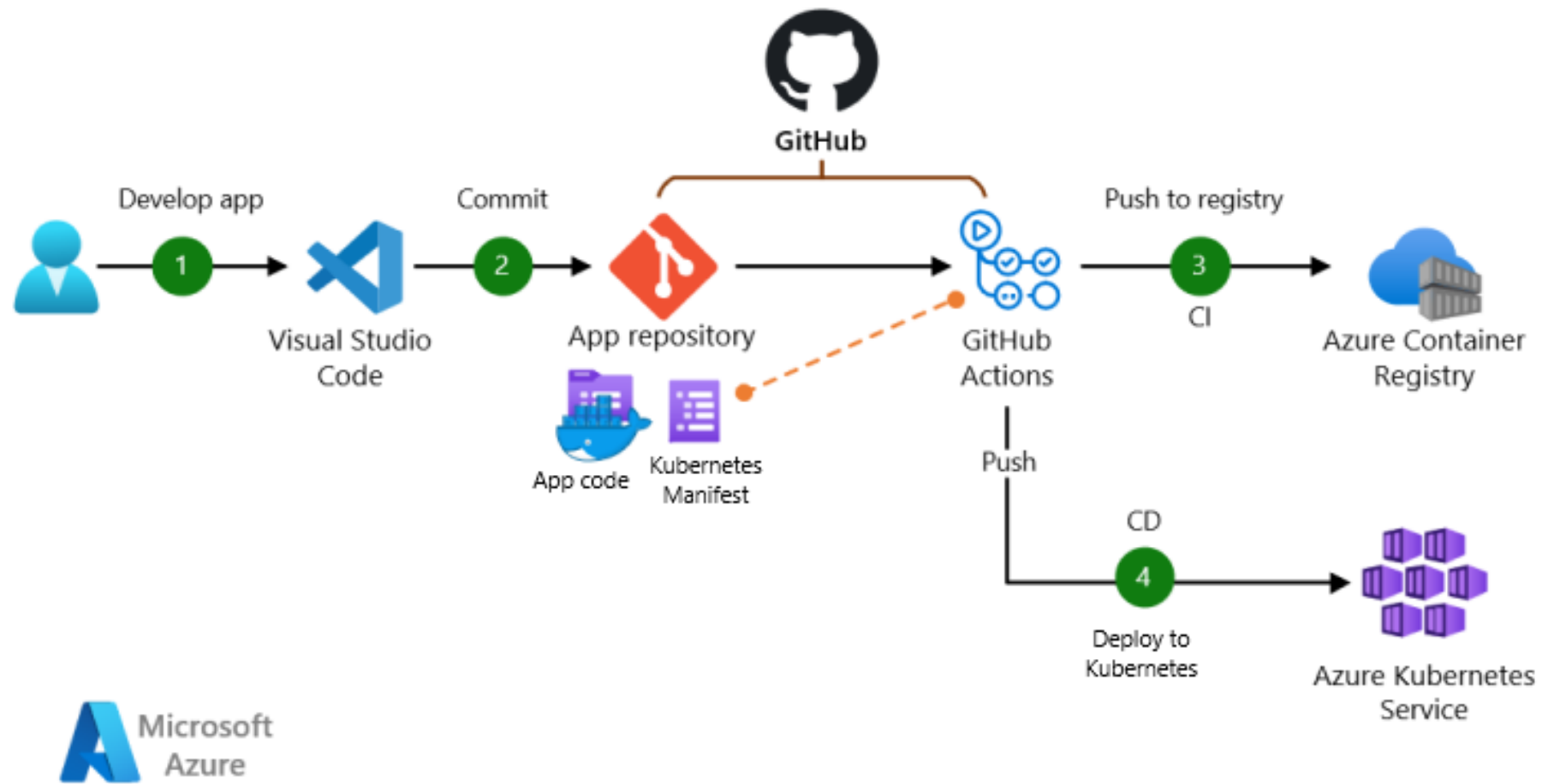
2. AI-augmented Software Engineering

It focuses on enhancing teams with AI technologies.

- **AI-Augmented Software Engineering Teams** - It advocates for making software engineering teams more efficient in their work using AI technology. Teams may swiftly develop a variety of artifacts using AI, such as design elements, application code, or test cases, which they can then change and reuse to speed up the overall process.
- **AI-Empowered Applications** - Automation of workflows, risk assessment, and the creation of models that suggest the optimal course of action are all capabilities of AI-enabled apps. Data-enhanced apps that improve business choices will be produced as a consequence of the convergence of readily available corporate data, cutting-edge model-building capabilities, and generative AI services.

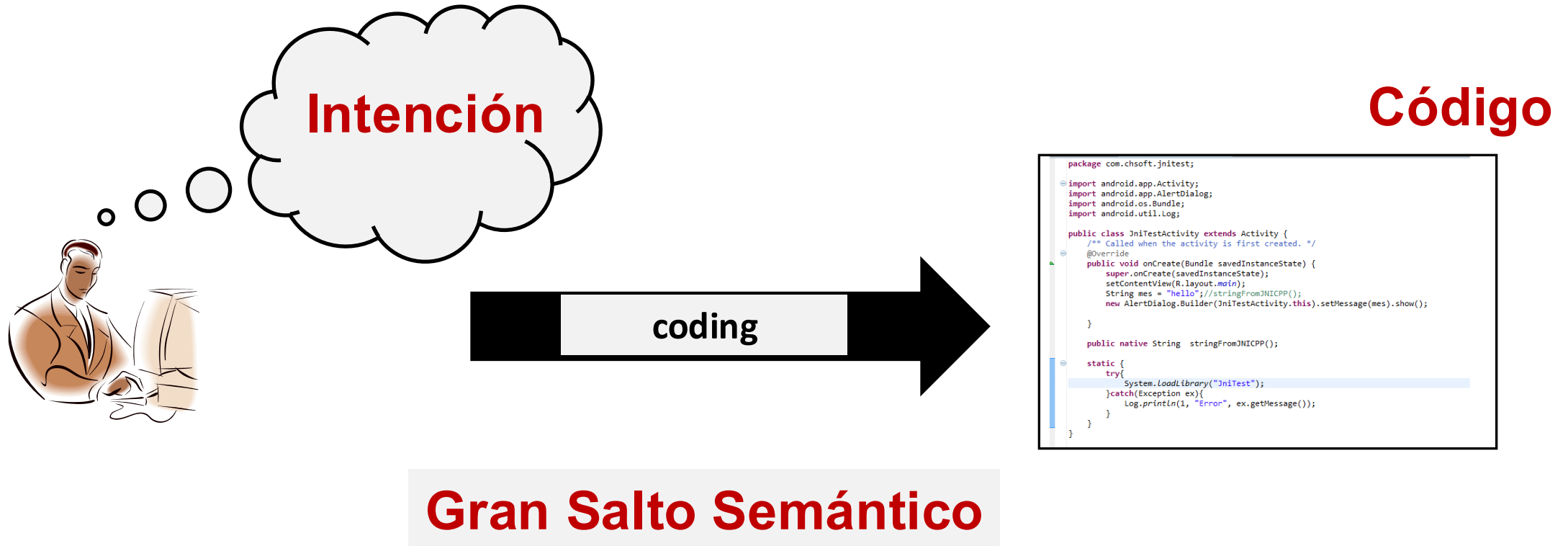
Desarrollo Full-Stack -> Front-end + Back-end



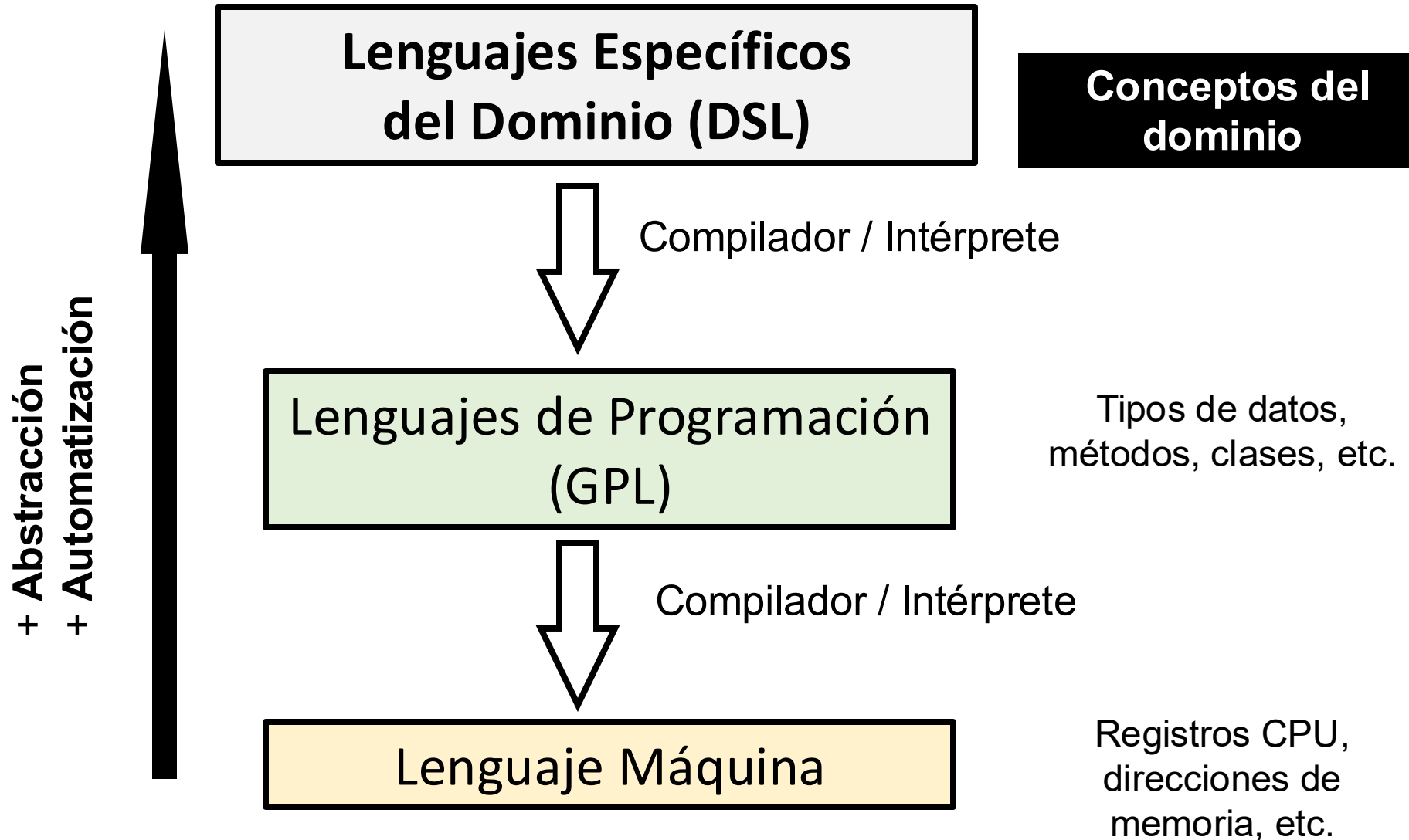


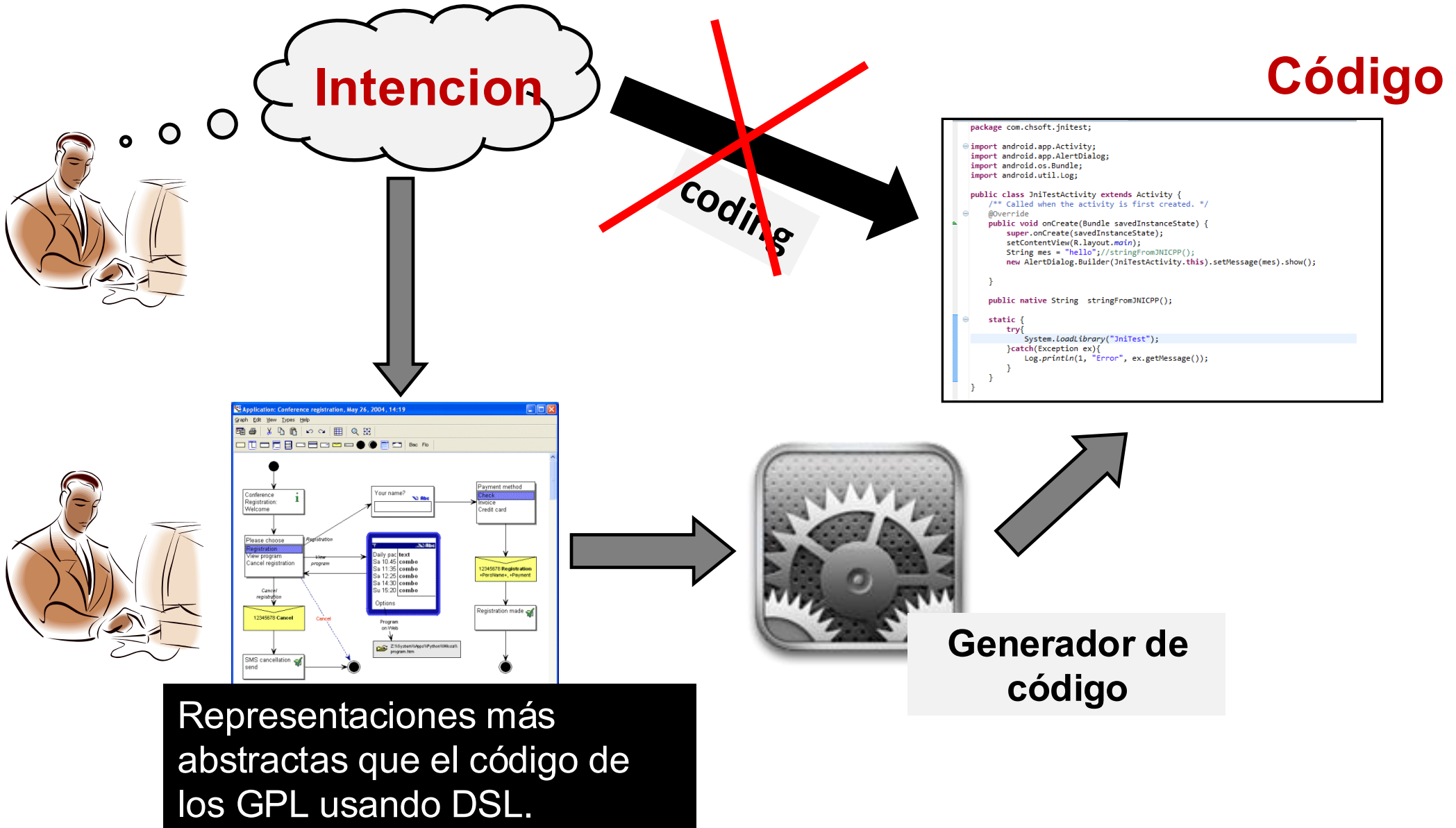
DevOps: un cambio en el código es automáticamente trasladado al usuario final

El gran problema en la construcción de software



Solución: Lenguajes más abstractos y generación de código





Tradicionalmente, la generación de código ha consistido en **escribir programas que escriben programas.**

Ahora, la **Generative AI** ha emergido, y el Código puede ser generado a partir de especificaciones expresadas en lenguaje natural o incluso imágenes.





Intención
(lenguaje natural)



You

Me gustaría crear una ventana que mostrase una tabla con información sobre usuarios. Cada fila tendría 4 columnas: NIF, nombre y apellidos, edad, y salario. Cada vez que se seleccione una fila de la tabla se debería mostrar debajo de ella una foto del usuario junto a información sobre el contrato con la empresa: fecha y categoría. Se supone que la clase Usuario tiene los campos nif, nombre fecha de nacimiento y salario, y otro campo contrato ¿Podrías crear esta ventana?

Intención

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.List;

public class VentanaUsuarios extends JFrame {
    private JTable tablaUsuarios;
    private JLabel labelFoto;
    private JLabel labelInfoContrato;
    private List<Usuario> usuarios;

    public VentanaUsuarios(List<Usuario> usuarios) {
        this.usuarios = usuarios;

        // Configuración general de la ventana
        setTitle("Información de Usuarios");
        setSize(800, 600);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        // Crear tabla
        String[] columnas = {"NIF", "Nombre y Apellidos", "Edad", "Salario"};
        DefaultTableModel model = new DefaultTableModel(columnas, 0);
        tablaUsuarios = new JTable(model);

        // Añadir datos de usuario a la tabla
        for (Usuario usuario : usuarios) {
```

ChatGPT for Software Engineers

Published by [O'Reilly Media, Inc.](#)

■ ■ Beginner to intermediate

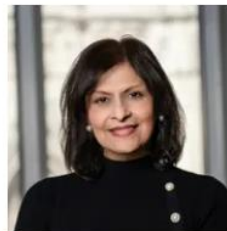
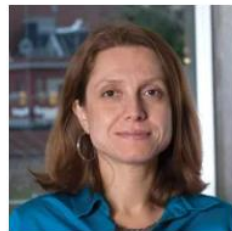
How to 10X your productivity with generative AI

[What you'll learn](#) [Is this live event for you?](#) [Schedule](#)

Course outcomes

- Understand the basics of ChatGPT and the importance of a good prompt
- Examine the vulnerabilities and risks of using ChatGPT
- Discover software development use cases for ChatGPT that save 50% of your time

Application of Large Language Models (LLMs) in Software Engineering: Overblown Hype or Disruptive Change?



IPEK OZKAYA, ANITA CARLETON, JOHN E. ROBERT, AND DOUGLAS SCHMIDT
(VANDERBILT UNIVERSITY)

OCTOBER 2, 2023

SEI Blog

Will Generative AI Kill Developer Jobs?



2024-03-21



HOLLY CUMMINS



SOFTWARE DEVELOPMENT

IA Generativa

Basada en Large-Language Models (LLM)

Desarrollo de software apoyado en IA

Aplicaciones potenciadas por la IA
(Atención médica, finanzas, etc.)

- **IA no reemplazará** a los desarrolladores de software, sino que los asiste.



GitHub
Copilot

- **Desarrollar con IA NO significa ser expertos en IA**

Índice de Contenidos

- Ingeniería del Software
 - Definición y Evolución
 - Importancia en la actualidad
- **Ciclo de vida del Desarrollo del Software (SDLC)**
 - Etapas y Modelos del SDLC
- Requisitos Software
 - Definición y tipos
 - Casos de uso e Historias de Usuario
- Modelado del Software con UML
 - Modelado de la estructura: Diagramas de clase
 - Modelado de las interacciones: Diagramas de Secuencia y Comunicación

Ciclo de Vida del Desarrollo del Software (SDLC)

Proceso sistemático y organizado en etapas utilizado para desarrollar software, destinado a asegurar la calidad y productividad (reducir tiempo y esfuerzo para reducir costes).

Etapas

Captura de Requisitos, Análisis y Diseño del Sistema, Implementación, Testing, Despliegue, y Mantenimiento/Operación

Modelos (Procesos):

Cascada, Incremental, Espiral, RAD, Ágil

Captura de Requisitos

- Identificación y especificación de requisitos: Funcionales y No Funcionales.
- Técnicas: Entrevistas, encuestas, prototipos.
- Obtención del documento SRS (Software Requirements Specification).

Análisis / Diseño del Sistema

- Estudio de viabilidad técnica, económica, legal.
- Diseño de la arquitectura del sistema, por ejemplo, una arquitectura en capas.
- Diseño de algunos aspectos claves del sistema: diagramas de entidades del dominio, esquema de la base de datos.

Implementación

- Escribir código de cada elemento software del sistema: clases, HTML, XML, JSON, ..
- Buenas practicas de programación, como patrones de código y patrones de diseño.
- Herramientas de construcción de sistemas (build tools) como Maven y sistemas de control de versiones como Git.

Despliegue

- Pasar el software del entorno de desarrollo al entorno de producción o entorno real en el que los usuarios finales pueden interactuar con el sistema creado.
- Varias estrategias: todo-a-la-vez (big bang), por fases, proyecto piloto, ..
- Herramientas actuales para automatizar despliegue: Docker, Kubernetes, Ansible, ..

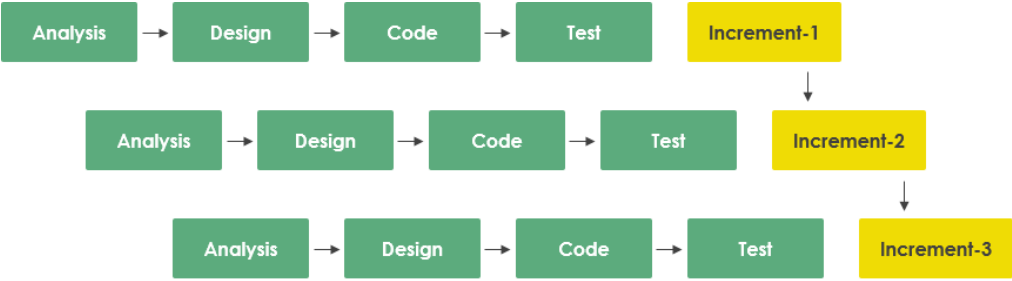
Testing

- Pruebas unitarias e integración continua.
- Pruebas del Sistema: interfaz de usuario, almacenamiento, seguridad, escalabilidad, ..
- Herramientas de automatización de pruebas: Junit, Jenkins, Selenium, Github Actions, ...
- Calidad $\neq$ Testing

Mantenimiento y Operaciones

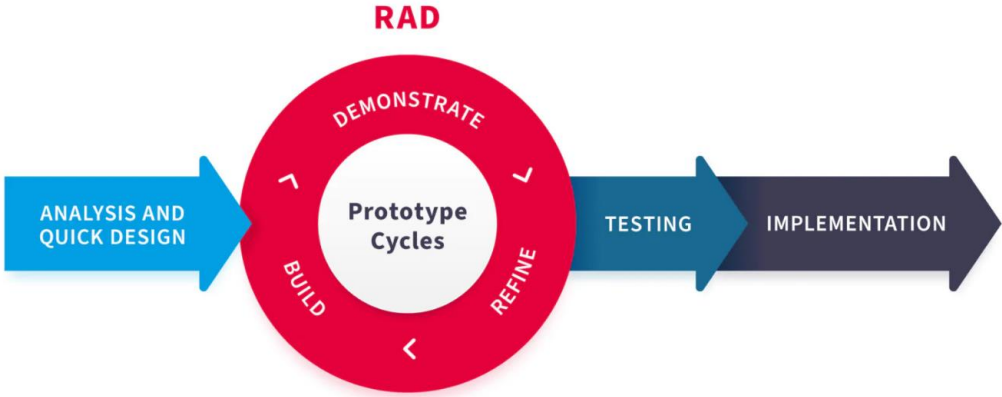
- Tipos de mantenimiento: Correctivo, predictivo, adaptativo, y perfectivo.
- Gestión de cambios y actualizaciones.

Modelos de Ciclo de Vida

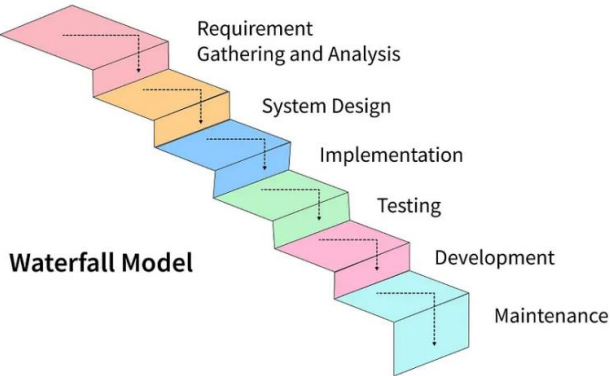


Incremental Model

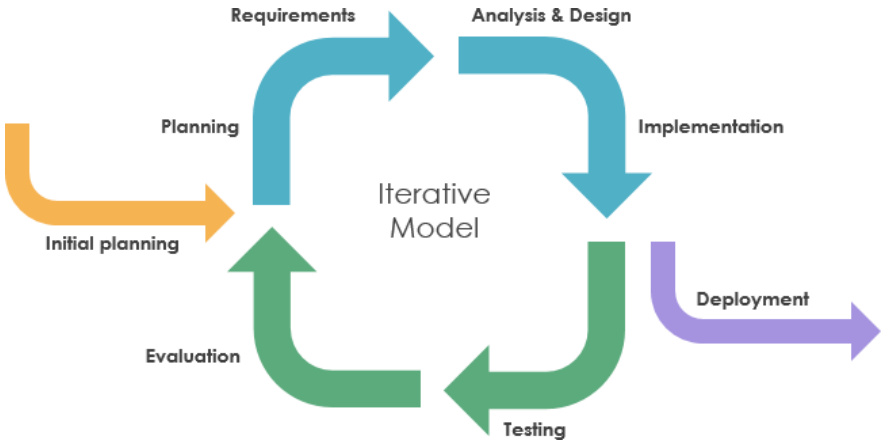
INCREMENTAL



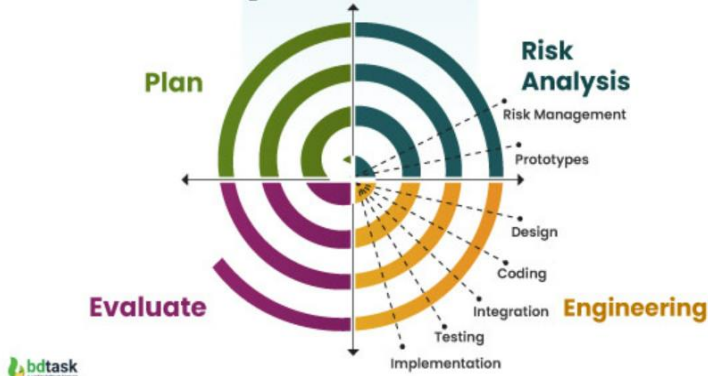
PROTOTIPADO



CASCADA



ITERATIVO



ESPIRAL

Métodos pesados y ágiles

- Métodos tradicionales eran “**pesados**”
 - En los 70's y 80's: Análisis y Diseño Estructurado, Yourdon, y con UML: RUP.
 - Muchos artefactos (modelos) creados en un ambiente burocrático
 - Muchas actividades realizadas con rigidez y control
 - Planificación detallada a largo plazo
 - Predictivos en vez de adaptativos
- Métodos ágiles
 - Da valor a las personas y relaciones entre ellas antes que al uso de procesos y herramientas.
 - Dedicar tiempo a trabajar con software antes que a realizar modelos y documentos.
 - Ser capaces de responder a los cambios es más importante que dedicar mucho tiempo a planificar
 - Importante colaborar con los clientes durante todo el proyecto

Extreme Programming, XP

Valores

- **Comunicación**
- **Simplicidad**
- **Realimentación**
- **Valentía** (frente al cambio)

Principios

- Aceptar el cambio
- Asumir Simplicidad
- Rápida Realimentación
- Cambio incremental
- **Trabajo de calidad**

Prácticas

- El juego de la planificación (*user stories*)
- Versiones pequeñas (desarrollo **iterativo e incremental**)
- Diseño simple
- Pruebas unitarias primero (**Test first**)
- **Refactoring**
- Programación por **parejas**
- **Código colectivo**
- **Integración continua**
- Semanas de 40 horas
- Cliente en el equipo
- **Estándares de codificación** (Patrones de código)

Índice de Contenidos

- Ingeniería del Software
 - Definición y Evolución
 - Importancia en la actualidad
- Ciclo de vida del Desarrollo del Software (SDLC)
 - Etapas y Modelos del SDLC
- **Requisitos Software**
 - Definición y tipos
 - Casos de uso e Historias de Usuario
- Modelado del Software con UML
 - Modelado de la estructura: Diagramas de clase
 - Modelado de las interacciones: Diagramas de Secuencia y Comunicación

Requisitos Software: Definición

Características y funcionalidades que un sistema debe tener para cumplir con las expectativas del usuario y las necesidades del negocio.

Requisitos Software: Tipos

- **Funcionales:** Definen las funciones específicas que el sistema debe realizar. **Qué debe hacer** el sistema para cumplir con su propósito.
 - Ejemplo: “*El sistema debe permitir a un usuario crear una cuenta usando su dirección de email*” o “*El sistema debe permitir a un usuario registrado enviar mensajes a otros usuarios con cuenta*”.
 - Formas de especificación más comunes: Casos de uso e historias de uso
- **No Funcionales:** Especifican los criterios de calidad del sistema como son rendimiento, seguridad, y usabilidad.

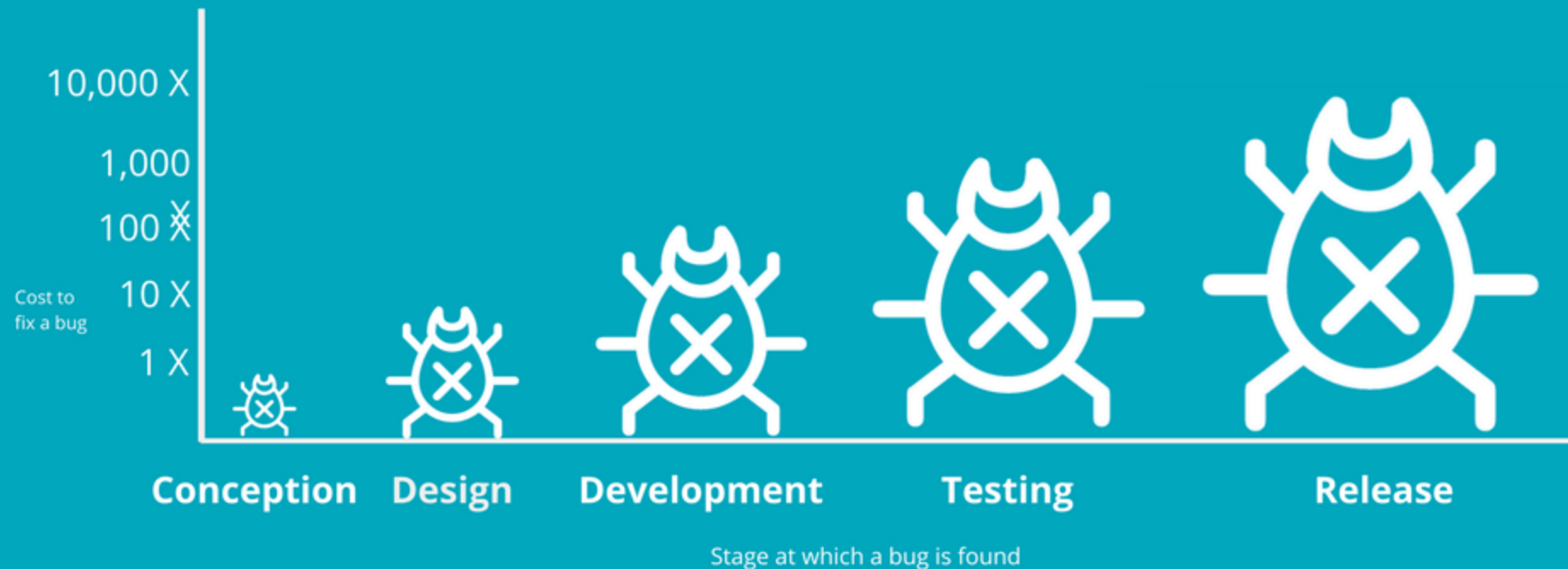
Requisito Software: Importancia

- Una incorrecta comprensión de los requisitos o que los requisitos “crecen” a lo largo del proyecto son una de las principales causas del fracaso de los proyectos software

Requisito Software: Importancia

- La correcta elicitación de requisitos es clave para el éxito del proyecto
- Cuanto más tarde se detecta un defecto mayor es el coste de corregirlo

Resolving bugs early and often reduces associated costs



Especificación de Requisitos Funcionales: Casos de Uso

Un **caso de uso** es una descripción detallada de cómo un sistema debe comportarse cuando un actor (un usuario o un sistema externo) interactúa con él para lograr un objetivo específico. Describen, paso a paso, las interacciones entre los actores y el sistema.

- **Actor Principal:** Usuario o sistema externo que interactúa con el sistema que se desea desarrollar para alcanzar un objetivo.
- **Escenario Principal (Camino Básico):** La secuencia principal de pasos que describe cómo el actor y el sistema interactúan para alcanzar el objetivo.
- **Extensiones (Caminos Alternativos):** Variaciones o escenarios alternativos que pueden ocurrir, incluyendo excepciones y errores.
- **Requisitos Especiales:** Criterios adicionales que el caso de uso debe cumplir, como restricciones de tiempo o limitaciones de seguridad.
- **Precondiciones y Postcondiciones:** Condiciones que deben ser verdaderas antes y después de que el caso de uso sea ejecutado.

Útiles para guiar la planificación de las iteraciones y realizar la validación con las pruebas de aceptación (acceptance tests)

- *Casos de Uso* en desuso desde la irrupción de los métodos ágiles (usan Historias de Usuario).

Especificación de Requisitos Funcionales: **Historias de usuario**

Una **historia de usuario** es una breve descripción escrita desde la perspectiva de un usuario final que captura una funcionalidad deseada del sistema. Las historias de usuario son una herramienta utilizada principalmente en metodologías ágiles, como Scrum, para capturar los requisitos de una manera sencilla y fácil de entender.

Características:

- **Formato Simple:** "Como [tipo de usuario], quiero [acción/funcionalidad] para [beneficio]."
- **Centrada en el Usuario:** Siempre se describe desde el punto de vista del usuario, destacando lo que quiere lograr y por qué.
- **Breve y Concisa:** Generalmente se mantiene simple y directa, sin detalles excesivos.
- **Criterios de Aceptación:** Se utilizan para definir cuándo una historia de usuario está completa, proporcionando una forma de verificar que se ha cumplido con lo requerido.

Ejemplo de requisito funcional: Historias de usuario (I)

Formato "Como [rol], quiero [funcionalidad] para [beneficio]".

Historias de Usuario para "Realizar Pedido en una Tienda Online de Ropa Deportiva"

Historia de Usuario 1: Navegación por el Catálogo de Productos

- **Como** cliente,
- **quiero** navegar por el catálogo de productos de ropa deportiva,
- **para** encontrar y seleccionar los productos que quiero comprar.

Historia de Usuario 2: Agregar Productos al Carrito de Compras

- **Como** cliente,
- **quiero** agregar productos a mi carrito de compras,
- **para** poder revisar y gestionar los artículos que deseo comprar antes de realizar el pedido.

Historia de Usuario 3: Realizar Pedido

- **Como** cliente,
- **quiero** confirmar mi pedido con los productos seleccionados en mi carrito,
- **para** completar mi compra y recibir los artículos en mi domicilio.

Requisitos No funcionales

- Los **requisitos no funcionales** (RNF) describen **cómo el sistema debe comportarse** y no qué debe hacer.
- Son **atributos de calidad** que definen las propiedades y restricciones del sistema, como el rendimiento, la seguridad, la usabilidad, etc.
- Determinan la calidad del sistema y la satisfacción del usuario final.
- Pueden impactar significativamente en la arquitectura, diseño, y experiencia de usuario del sistema.
- Los RNF son cruciales para la aceptación del producto, a menudo más que los requisitos funcionales.

Tipos de Requisitos No funcionales

- **Rendimiento:** Tiempo de respuesta del sistema, capacidad de carga, tiempo de procesamiento.
 - Ejemplo: "El sistema debe procesar 1000 transacciones por segundo."
- **Seguridad:** Autenticación, autorización, encriptación de datos, protección contra ataques.
 - Ejemplo: "El sistema debe soportar autenticación de dos factores."
- **Usabilidad:** Facilidad de uso, accesibilidad, consistencia de la interfaz.
 - Ejemplo: "El sistema debe ser accesible para personas con discapacidades visuales."
- **Fiabilidad:** Disponibilidad, recuperación de fallos, tolerancia a fallos.
 - Ejemplo: "El sistema debe tener una disponibilidad del 99.9%."
- **Mantenibilidad:** Modularidad, facilidad de actualización, documentación.
 - Ejemplo: "El código debe estar documentado al 100% con comentarios claros y concisos."
- **Portabilidad:** Compatibilidad con diferentes plataformas y sistemas operativos.
 - Ejemplo: "El sistema debe ser compatible con los navegadores más comunes."

Criterio de Aceptación: Métricas específicas que permiten verificar si el sistema cumple o no con un determinado requisito no funcional.

Ejemplo: "El tiempo de respuesta del sistema debe ser inferior a 2 segundos bajo una carga de 500 usuarios concurrentes."

Fases del Proceso de Ingeniería de Requisitos

Elicitación: Recopilación de requisitos a través de técnicas como entrevistas, encuestas y prototipos.

Especificación: Documentación detallada de los requisitos. Los requisitos deben ser:

- Claros, concisos y completos.
- Verificables, es decir, debe ser posible determinar si se han cumplido o no.
- Consistentes, es decir, no deben existir contradicciones entre ellos.
- Priorizados, ya que no todos los requisitos tienen la misma importancia.

Validación y Verificación: Asegurar que los requisitos sean correctos, completos y factibles.

Gestión de Requisitos: Manejo de cambios en los requisitos y mantenimiento de la trazabilidad.

Herramientas: Jira, Confluence, Trello,...

Índice de Contenidos

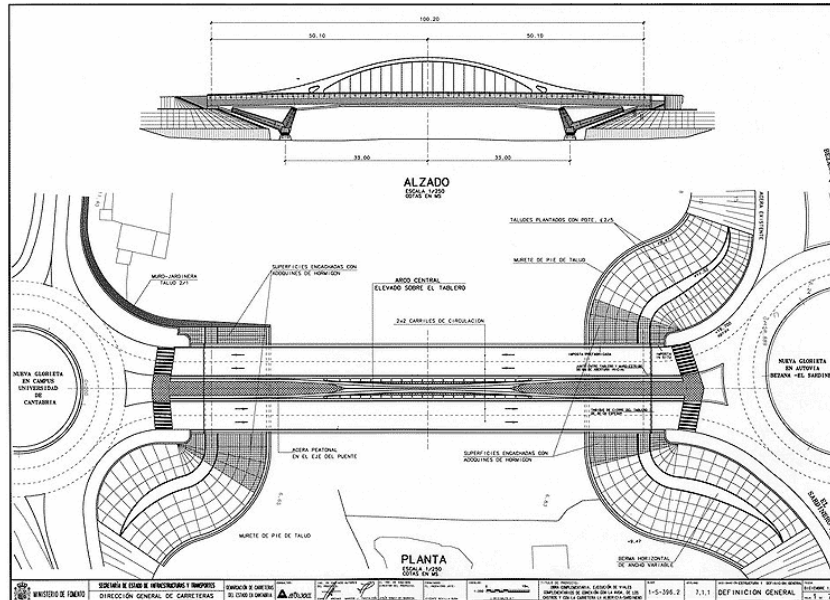
- Ingeniería del Software
 - Definición y Evolución
 - Importancia en la actualidad
- Ciclo de vida del Desarrollo del Software (SDLC)
 - Etapas y Modelos del SDLC
- Requisitos Software
 - Definición y tipos
 - Casos de uso e Historias de Usuario
- **Modelado del Software con UML**
 - Modelado de la estructura: Diagramas de clase
 - Modelado de las interacciones: Diagramas de Secuencia y Comunicación



Ceci n'est pas une pipe.

Noción de Modelo

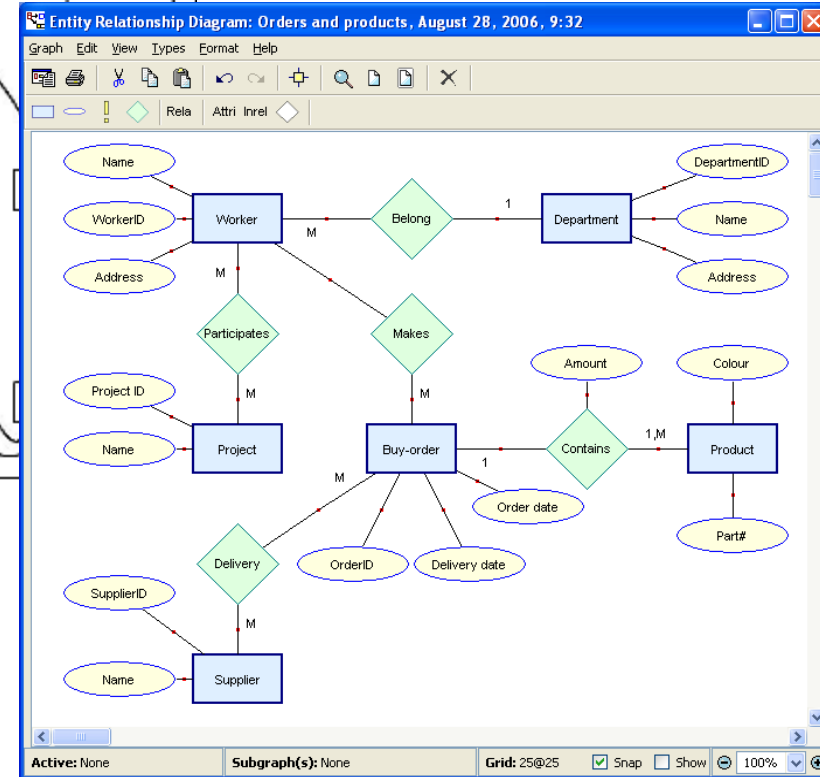
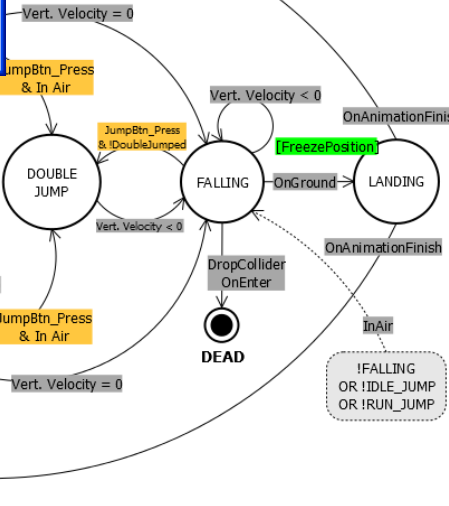
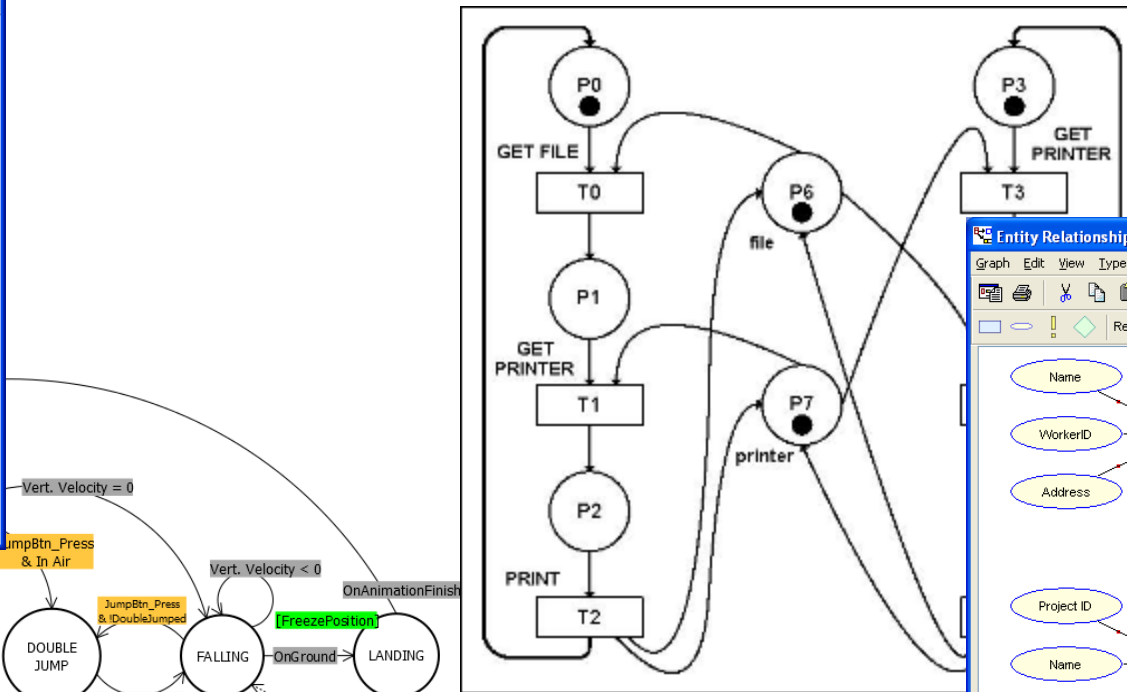
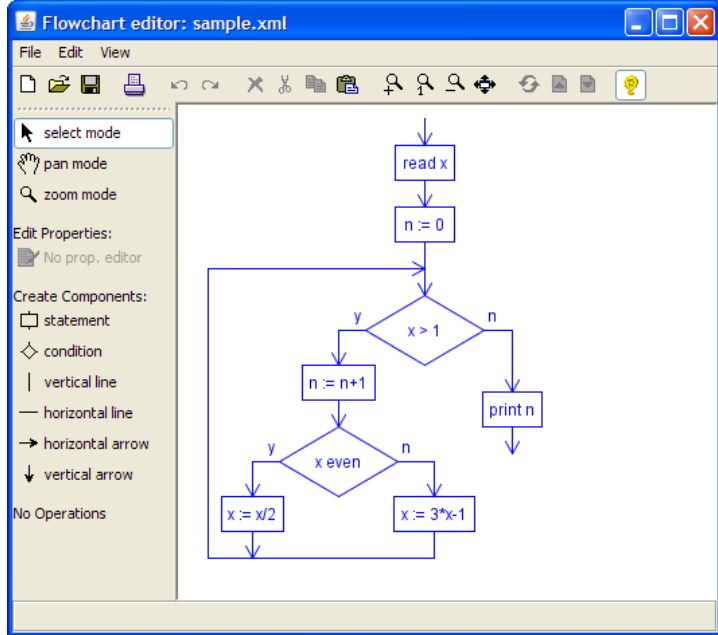
*“Un modelo es una **representación de la realidad** que es obtenida por medio de un proceso de **abstracción** con el propósito de ayudar a comprender y razonar sobre esa realidad”. Un modelo es expresado en un **lenguaje**.*



Proceso de Abstracción: Se ignoran los detalles y nos centramos en las características esenciales que nos interesan

Modelos del Software

Una descripción abstracta de algún aspecto de los sistemas software, por ejemplo: requisitos, estructura, comportamiento, datos, ..



El Lenguaje Unificado de Modelado, UML

- A mediados de los noventa existían muchos métodos de análisis y diseño OO: Mismos conceptos con distinta notación. **Mucha confusión**.
- En 1994, Grady Booch, James Rumbaugh e Ivar Jacobson deciden unificar las notaciones de sus métodos: **U**nified **M**odeling **L**anguage (UML).
- Proceso de estandarización promovido por el **OMG**.
 - Propone, elabora y mantiene especificaciones para aplicaciones empresariales distribuidas e interoperables.
 - Estándares OMG: Corba, UML, OCL, MDA, ..
- Ventajas de UML:
 - Eliminar confusión y proporcionar estabilidad al mercado.
 - Reunir puntos fuertes de cada método y añadir mejoras.



El Lenguaje Unificado de Modelado, UML

UML es un lenguaje para *visualizar, especificar, construir y documentar* los **modelos** de un sistema software, desde una perspectiva orientada a objetos.

Objetivos en su diseño:

- Modelar desde los requisitos al despliegue
- Modelar todo tipo de sistemas
- Utilizable por personas y ejecutable
- Equilibrio entre simplicidad y potencia expresiva

Diagrama de clases UML (estructura)

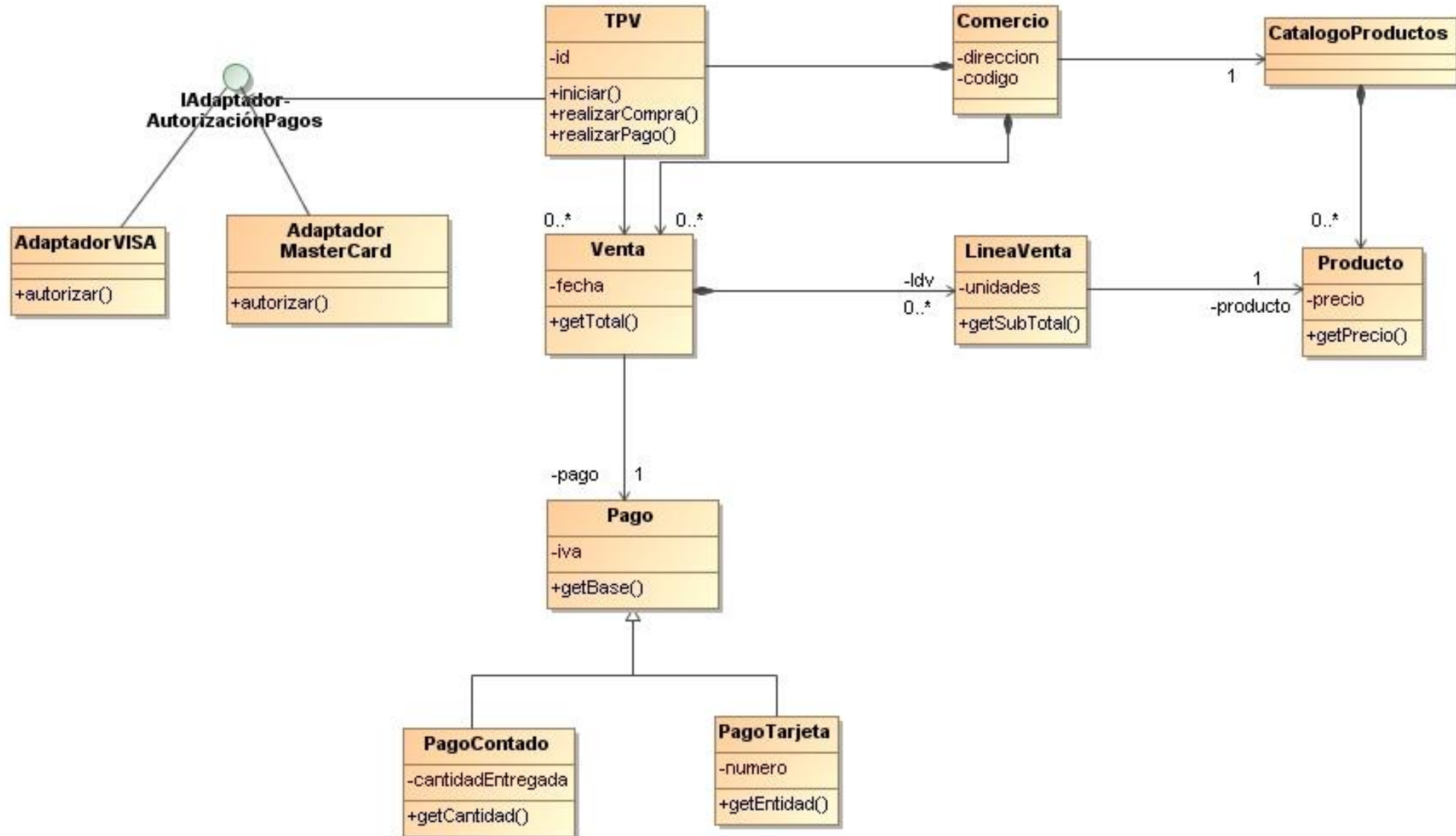
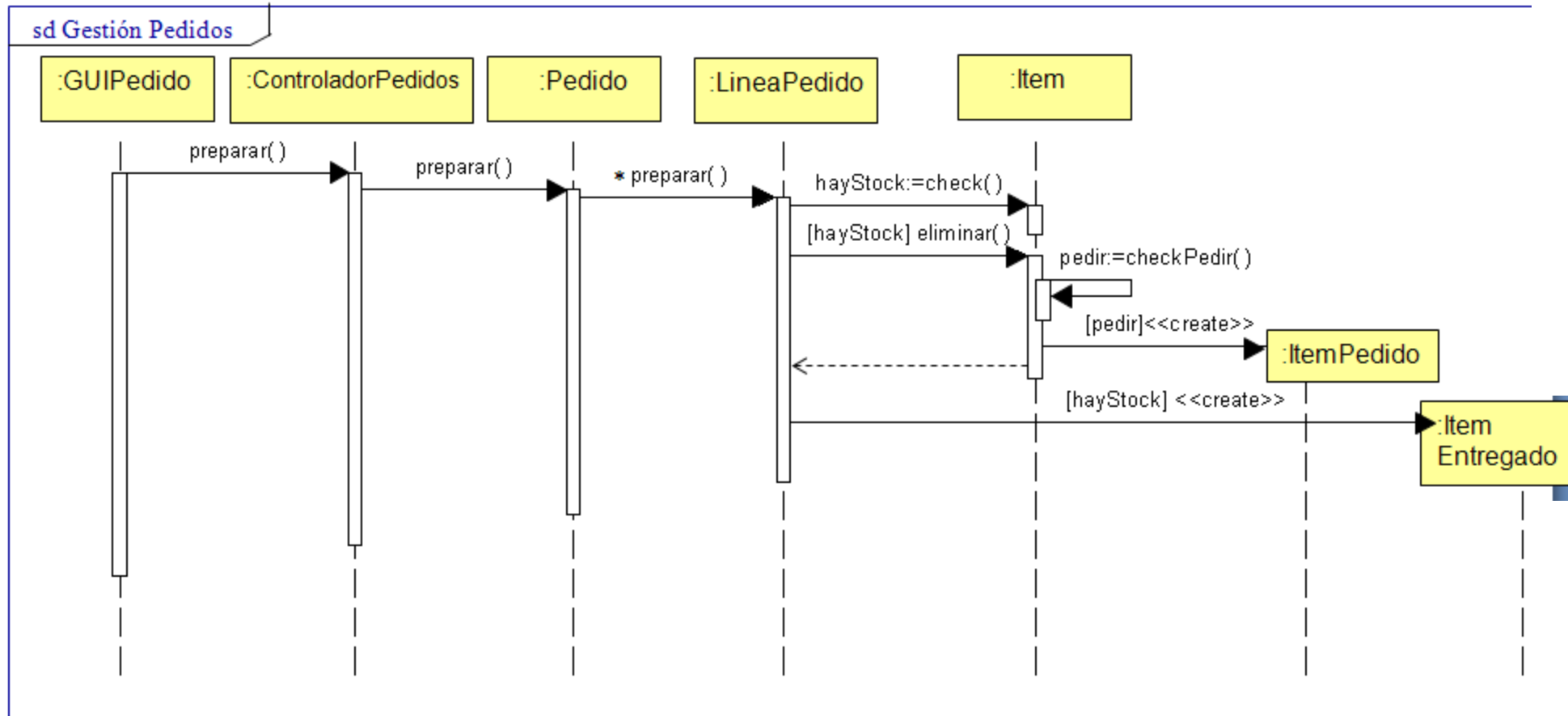


Diagrama de Secuencia UML (comportamiento)



¿Cuál es la utilidad de los modelos?



© Can Stock Photo - csp11667019

Documentación

(decisiones de diseño,
Implementación)

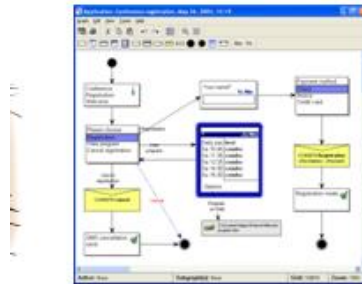


Comunicación (Visualización)

¿Cuál es la utilidad de los modelos?



Razonar
(Visualización)



Model



Generation Tools

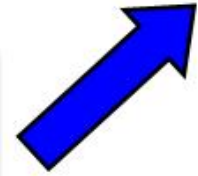
```
package com.charts.jstest;
import android.app.Activity;
import android.app.AlertDialog;
import android.os.Bundle;
import android.util.Log;

public class JstestActivity extends Activity {
    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);
        String msg = "Hello!!";
        new AlertDialog.Builder(JstestActivity.this).setMessage(msg).show();
    }

    public native String stringFromJNI();

    static {
        try {
            System.loadLibrary("Jstest");
        } catch (Exception ex) {
            Log.println(Log.ERROR, "Error", ex.getMessage());
        }
    }
}
```

Code



¡¡Generación de código!!

Modelo vs. Diagrama

- Importante distinción cuando se genera código a partir de modelos.

Un **modelo** es una **especificación completa y precisa** de un aspecto del sistema.

Un **diagrama** representa una parte de la información del **modelo** en forma gráfica o textual.

Por ejemplo, un diagrama de clases no suele incluir todas las clases, atributos, métodos y asociaciones del sistema bajo estudio, sino aquellas que se consideran relevantes para el propósito que se desea transmitir.

Uso actual del modelado UML

Grady Booch, uno de los 3 creadores de UML

“Sólo deberíamos usar diagramas para aquellas cosas sobre las que no se puede razonar sobre el código”

When we began with the UML, **we never intended it to become a programming language**. We never got the notation for collaborations right. Component and deployment diagrams needed more maturing. The UML metamodel became grossly bloated, because of the drive to model driven development. I think that complexity was an unnecessary mistake.

Seriously, **you need about 20% of the UML to do 80% of the kind of design you might want to do in a project – agile or not** – but use the UML with a very light touch: use the notation to reason about a system, to communicate your intent to others...and then throw away most of your diagrams.

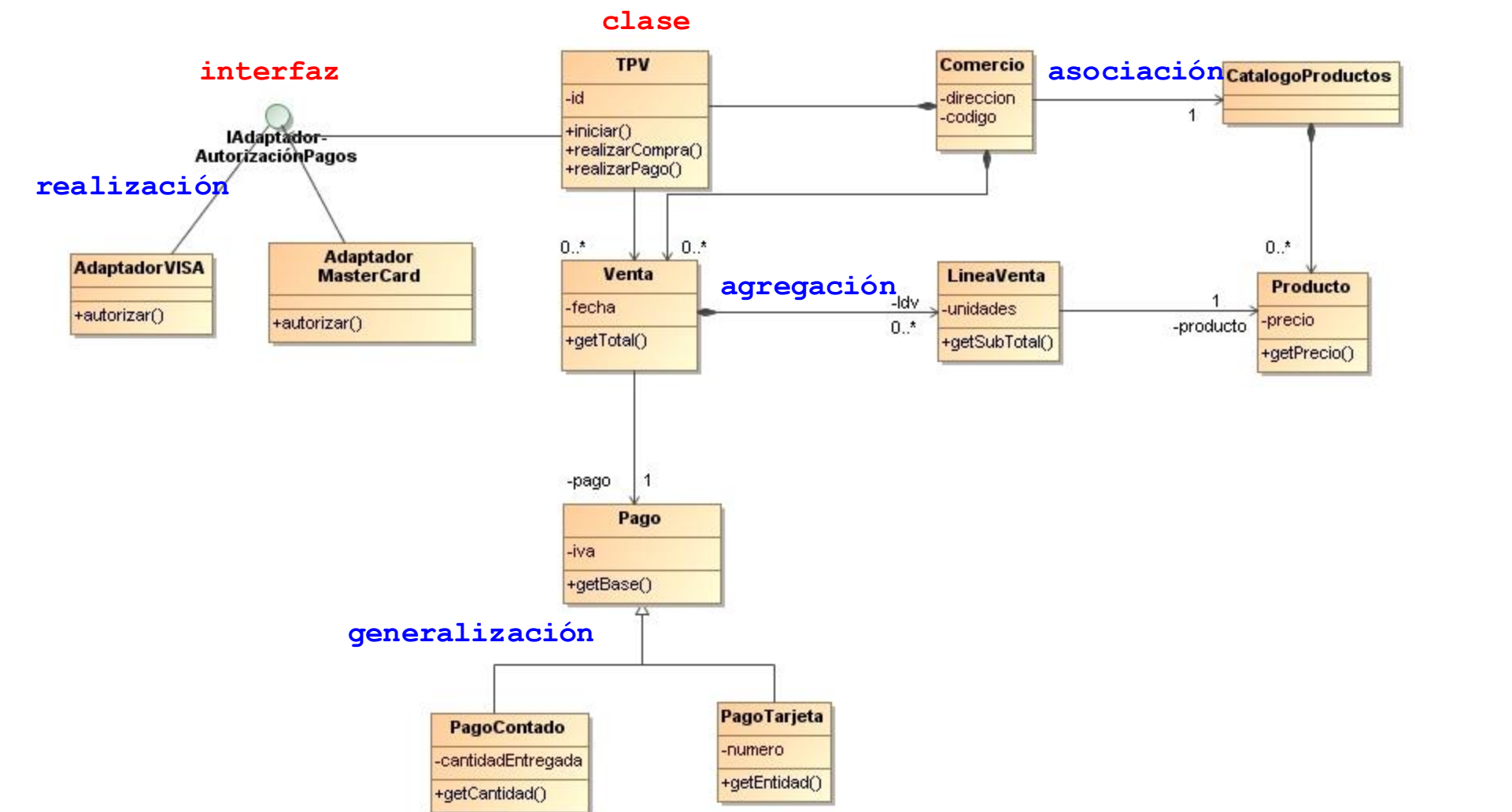
Modelos UML usados en la actualidad

- ~~Diagramas de casos de uso~~ para requisitos (No se usan)
- **Modelo Estructural de clases del dominio: Diagramas de clases**
- **Modelo Comportamiento:**
 - Diagramas de interacción (principalmente para documentación)
 - Máquinas de estado: Diagramas (Máquinas) de estado
 - Diagramas de actividades (**workflows de procesos de negocio**, herramientas BPMN)

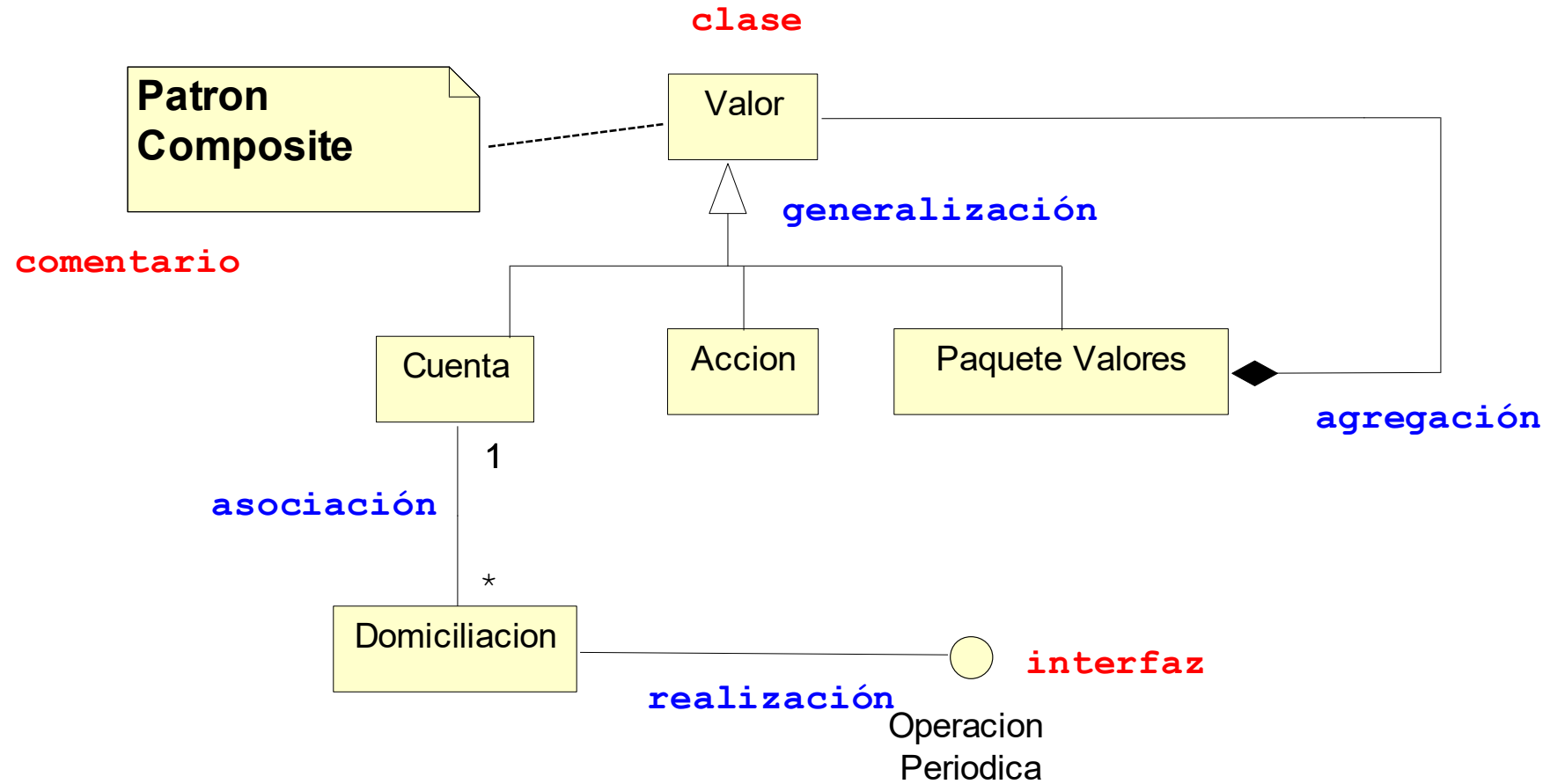
Modelado estructural: Diagrama de clases

- Se describen los **tipos de objetos** de un sistema y las **relaciones** que existen entre ellos.
 - Clases, Interfaces, Relaciones de dependencia, realización, generalización y asociación
- Un **diagrama de clases** es una representación gráfica de un modelo estructural.
- Diferentes usos: modelo **conceptual**, modelo de clases del **diseño / implementación**.
- En TDS modelo de implementación: clases del negocio o dominio de la aplicación.

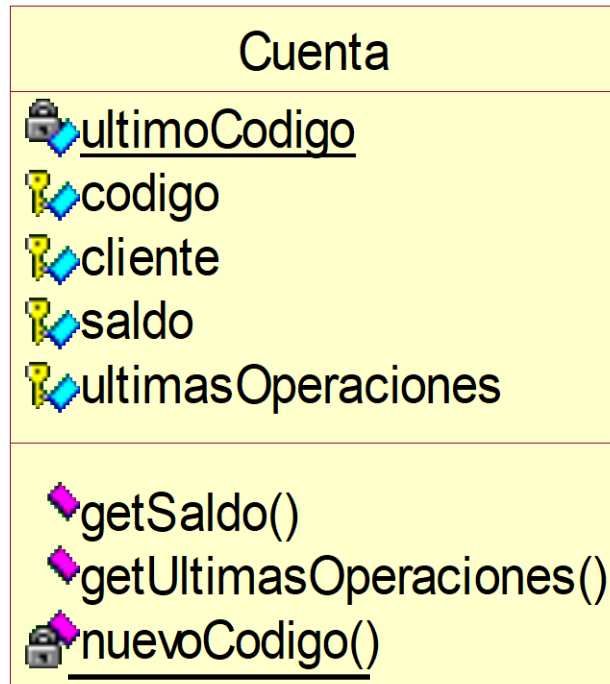
Ejemplo de diagrama de clases



Ejemplo de diagrama de clases

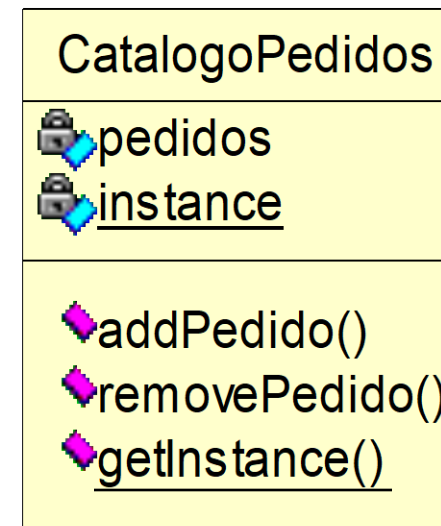
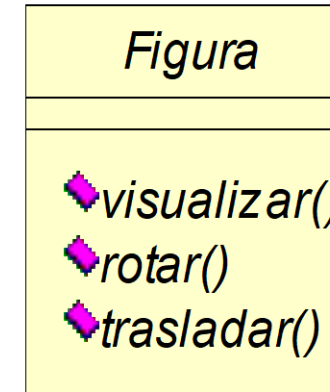


Clases



Atributos

Operaciones



- Clases y métodos abstractos (cursiva)
- Variables y métodos de clase (subrayado)

Atributos

[visibilidad] nombre [: tipo] [['multiplicidad']]

visibilidad	{	+	=	pública	nombre: nombre del atributo tipo: tipo del atributo
		#	=	protegida	
		-	=	privada	
		~	=	package	

Operaciones

[visibilidad] nombre ['('lista_parametros')'] [: tipo_retorno]

visibilidad	{	+	=	pública	nombre: nombre de la operación
		#	=	protegida	lista_parámetros: separados por comas
		-	=	privada	tipo retorno:
		~	=	package	

Atributos y Operaciones. Ejemplos

origen

+origen

origen : Punto

nombre : String [0..30]

origen : Punto = (0,0)

id : Integer {readOnly}

dibujar()

+dibujar()

set (nombre : String)

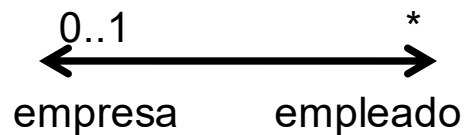
getID() : Integer

Relaciones

Dependencia
(no se suelen usar)



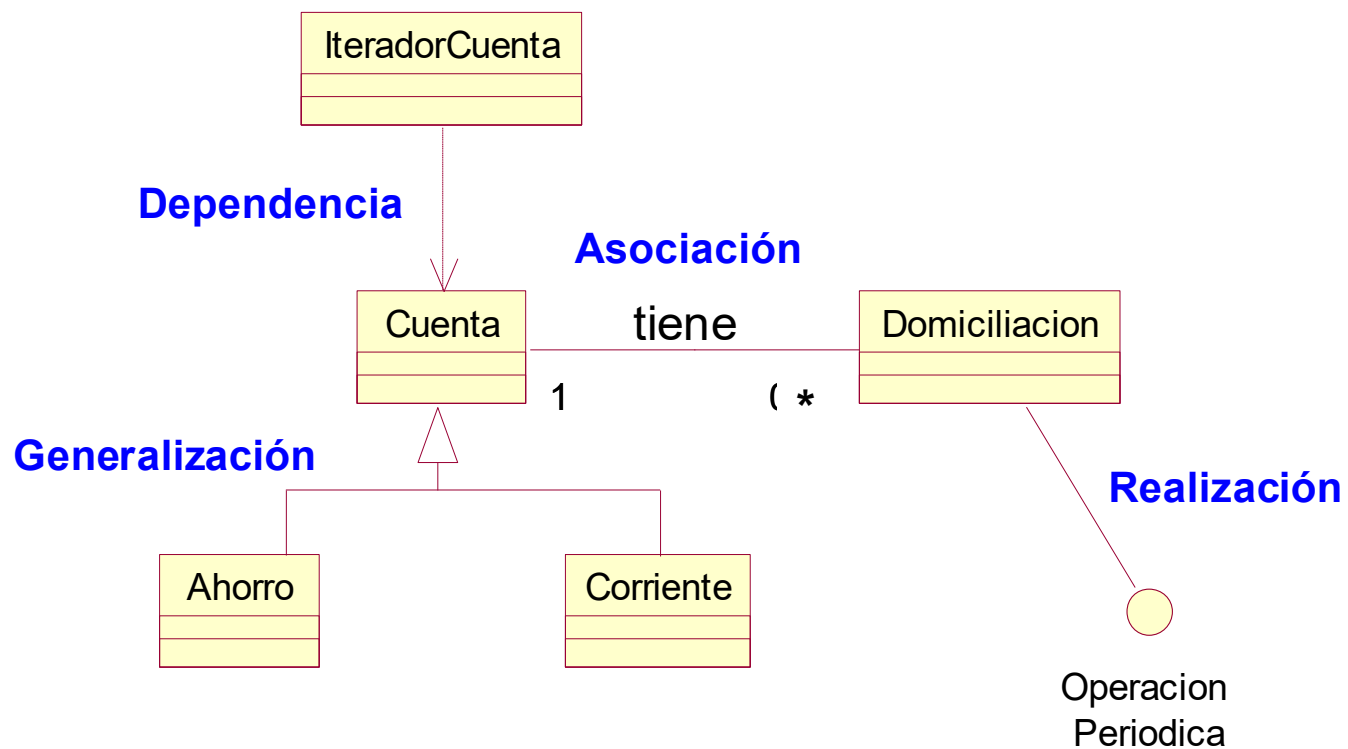
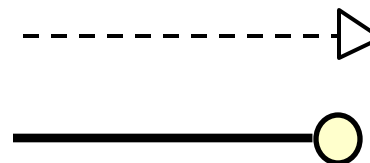
Asociación



Generalización

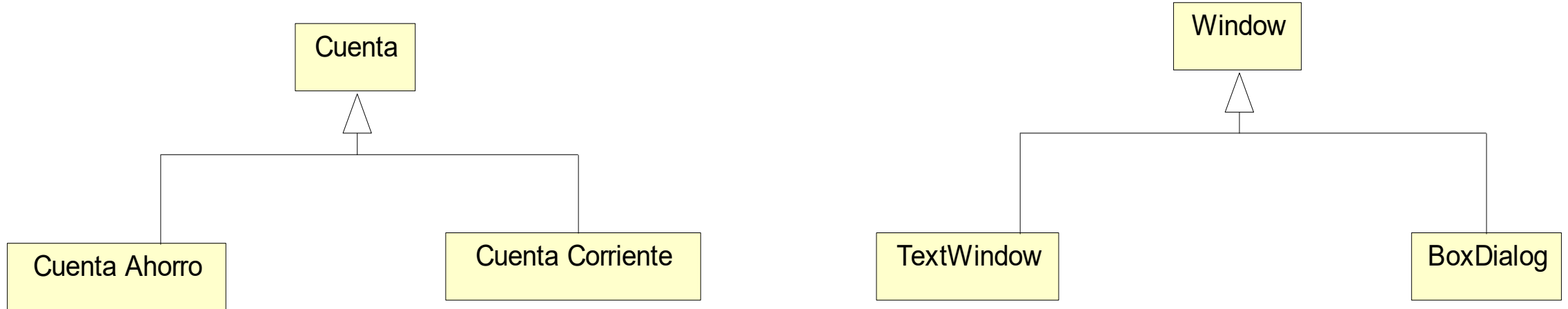


Realización



Generalización

- *Relación “Es-un-tipo-de”*
- En el caso de un modelo de diseño o implementación denota una relación de *herencia*



Asociación

- Relación estructural que especifica que los objetos de un tipo están conectados con los de otro tipo.

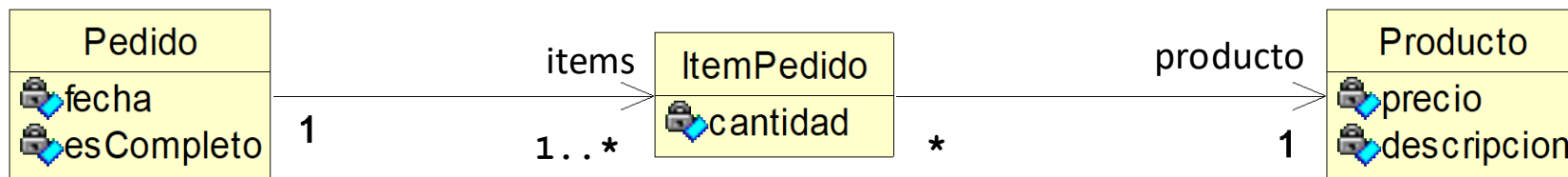


Multiplicidad (mínimo..máximo) {
0..1, 1, 0..*, *, 1..* (comunes)
1..10, 2..25 (no comunes)

Visibilidad {
Pública: + empresa
Protegida: # empresa
Privada: - empresa
Paquete: ~ empres

Asociación: Navegabilidad

- Asociaciones son **bidireccionales** pero es posible restringir la **navegación** de una asociación a una sola dirección.
- Determina si una clase de un extremo de la asociación tiene “conocimiento” de la clase del otro extremo.
- Se utiliza en diagramas de diseño e implementación

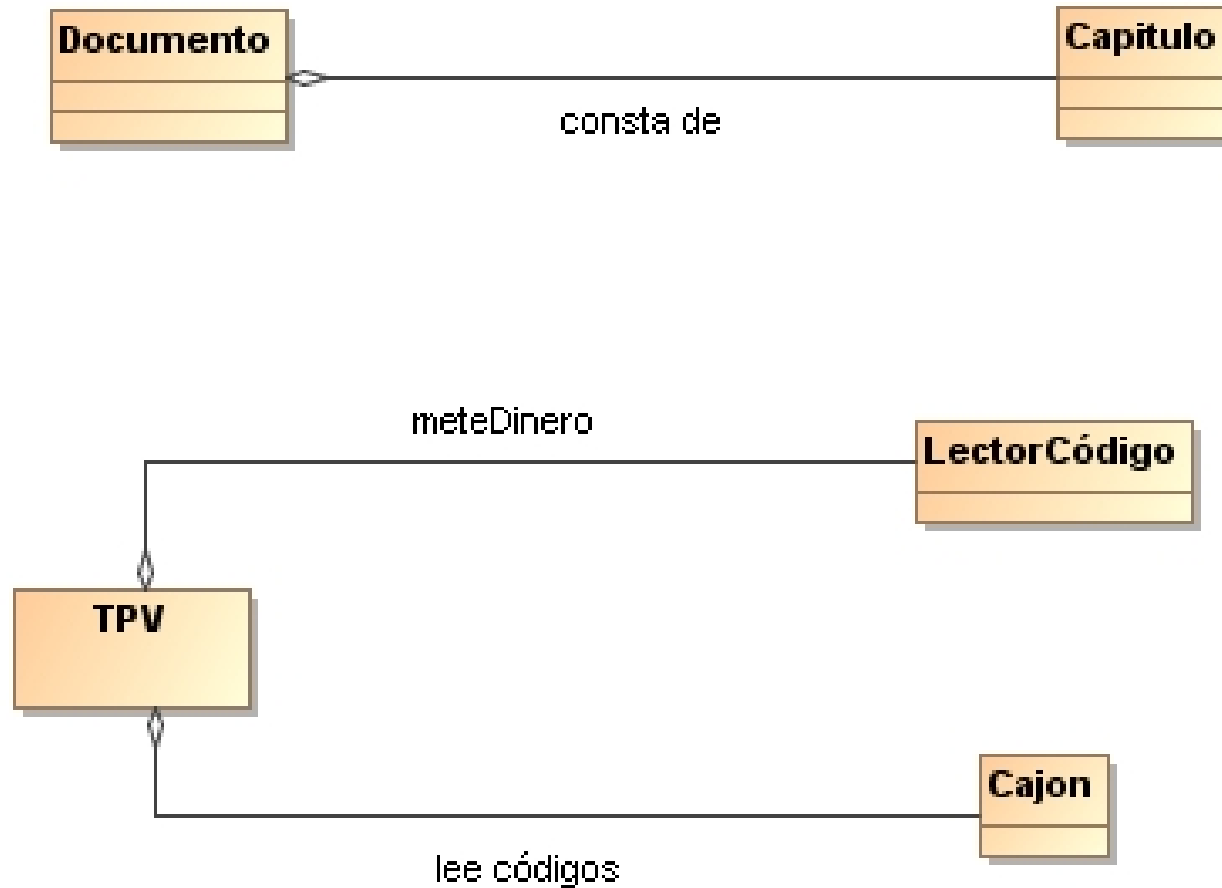


```
class Pedido {..  
    private ArrayList<ItemPedido> items;  
    private Date fecha;  
    private boolean esCompleto;  
}
```

```
class ItemPedido {..  
    private Producto producto;  
    private int cantidad;  
}  
  
class Producto {..  
    private Money precio;  
    private String descripcion;  
}
```

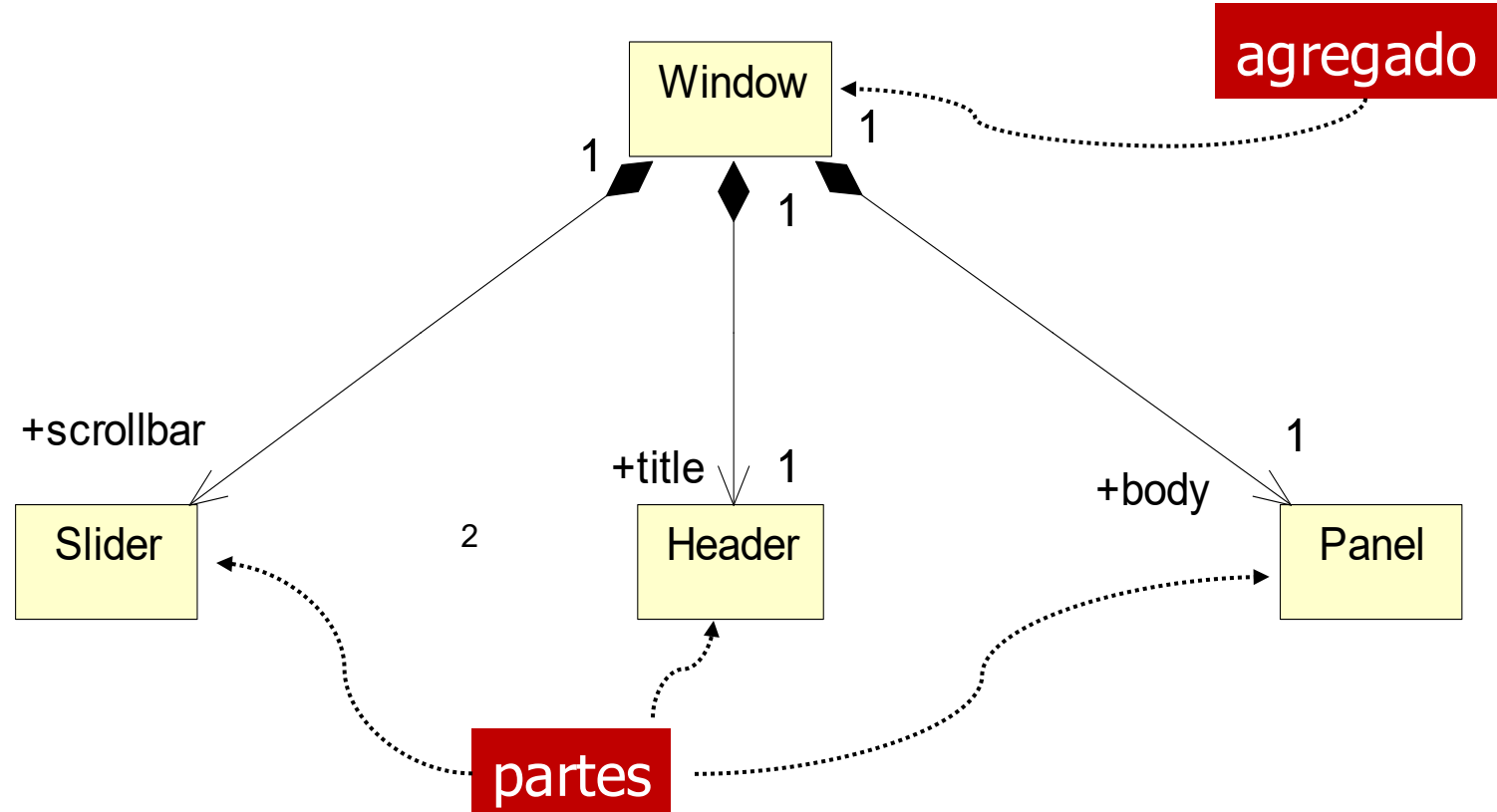
Agregación

- Caso especial de asociación que expresa la **relación parte-de**: *un objeto es parte de otro objeto (agregado)*



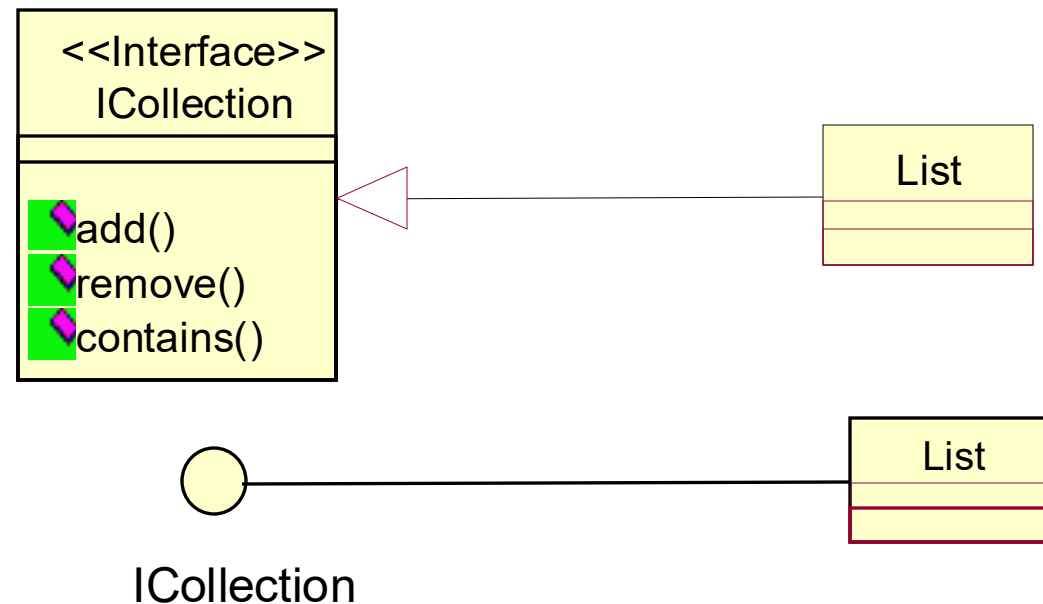
Composición

- Un tipo especial de agregación:
 - Una parte pertenece a un único agregado (**exclusividad**)
 - Si se elimina un agregado se eliminan todas sus partes (**dependencia**)
- Una parte se puede añadir/eliminar en cualquier instante.



Realización

- Una **interfaz** es una colección de operaciones que especifican los servicios de una clase o componente.
- Una interfaz permite separar la especificación de la implementación.
- Una **realización** especifica que una clase implementa una interfaz



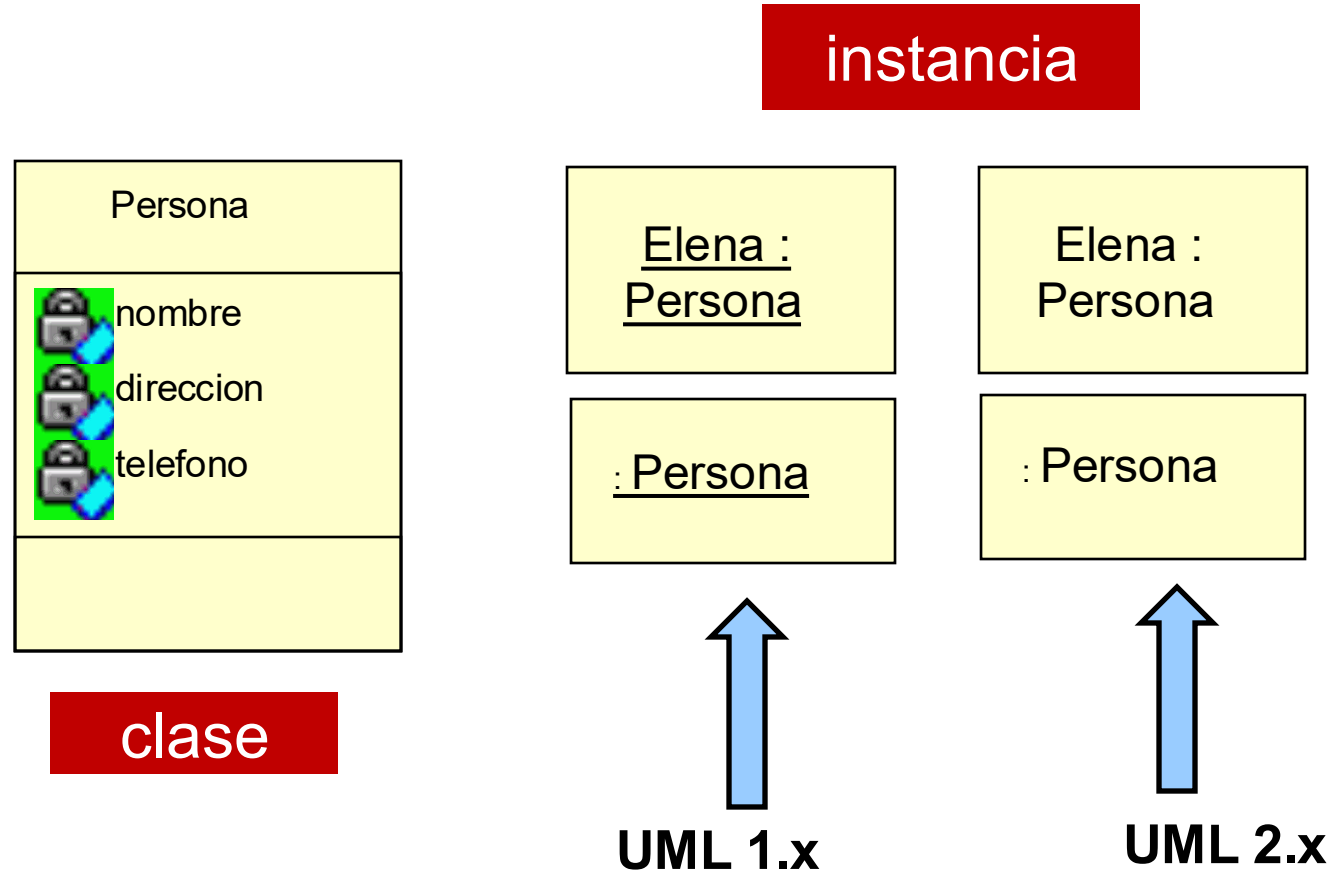
Modelado del comportamiento

- Diagramas de interacción describen cómo los objetos colaboran entre sí para realizar cierta actividad (TDS).
 - Diagramas de **Secuencia** y Diagramas de **Colaboración** o Comunicación

Diagramas de Interacción. Definiciones

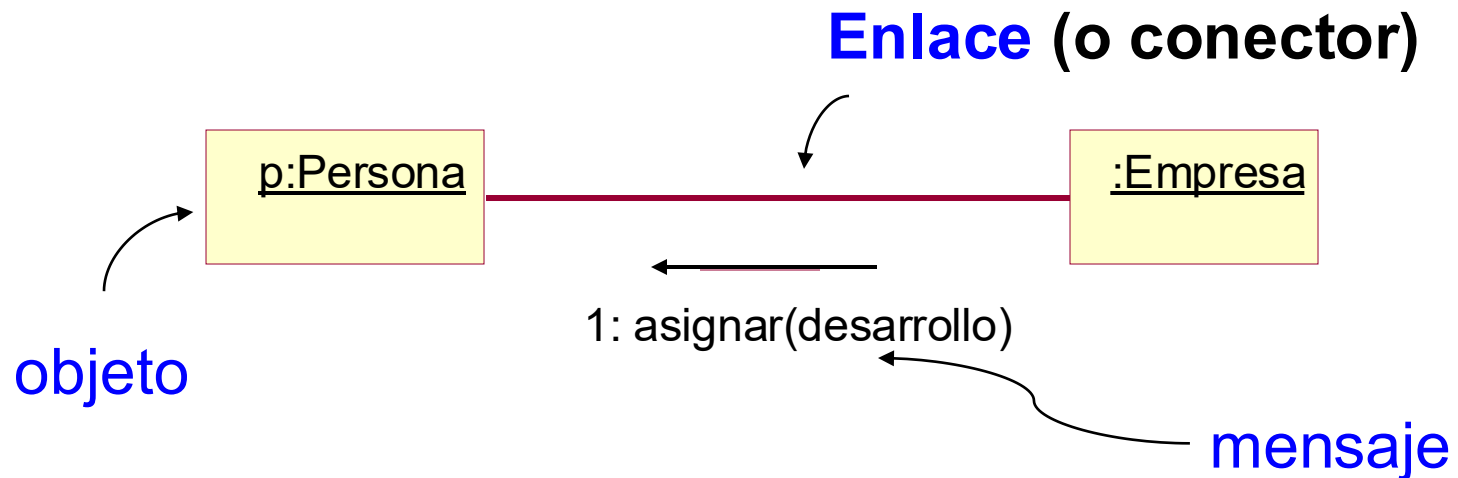
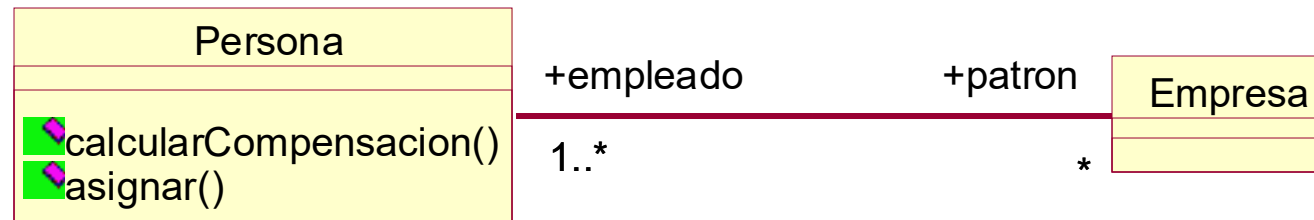
- **Interacción**: Comportamiento que comprende un conjunto de **mensajes** intercambiados entre un conjunto de **líneas de vida** (o a través de un **enlace**) para lograr un propósito.
- **Mensaje**: especificación de una particular comunicación entre líneas de vida de una interacción que transmite información, con la expectativa de desencadenar una actividad.
- **Línea de Vida**: Objeto y la línea de tiempo que marca su existencia (diagrama de secuencia) o enlace (diagramas de comunicación).

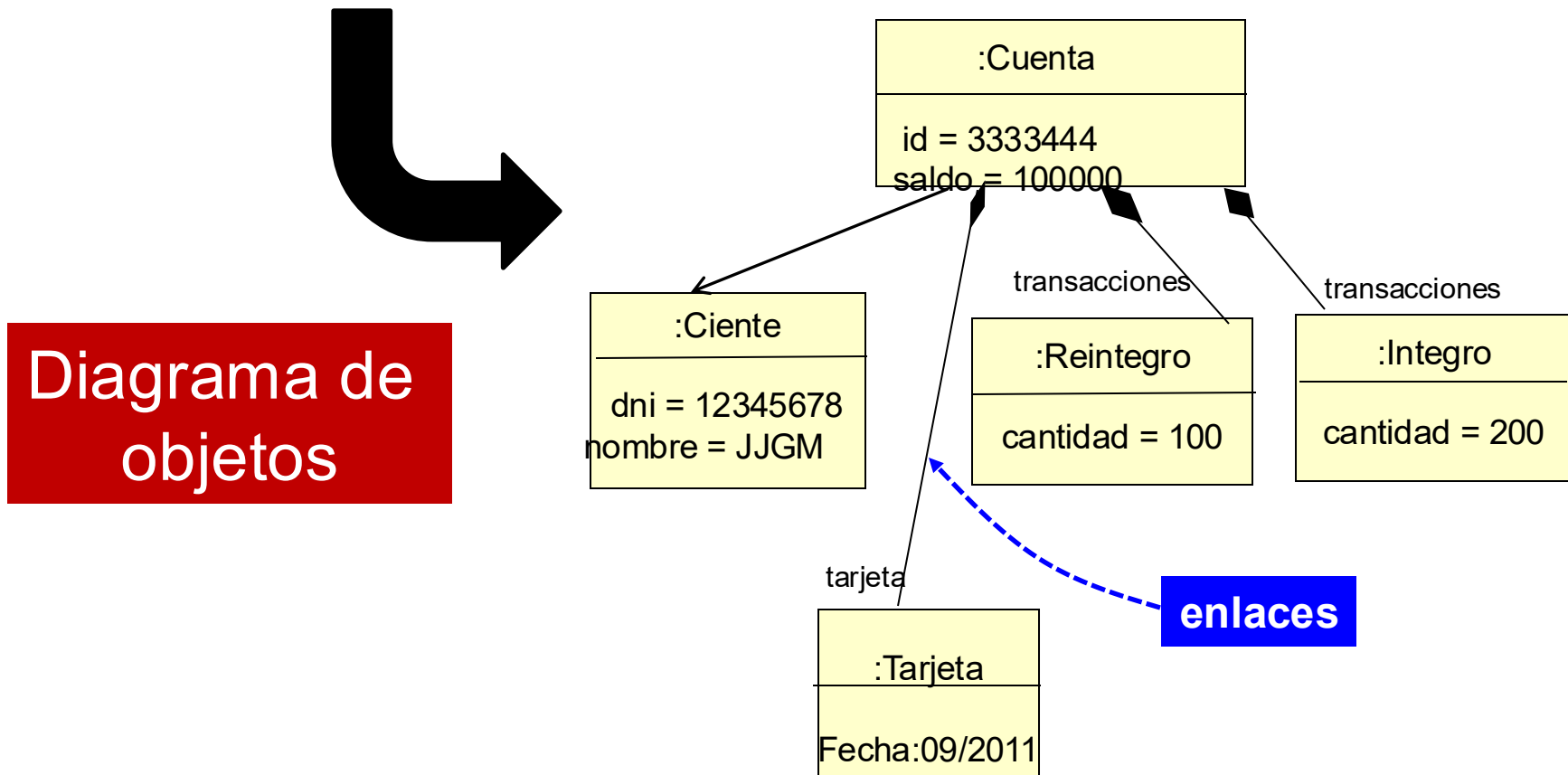
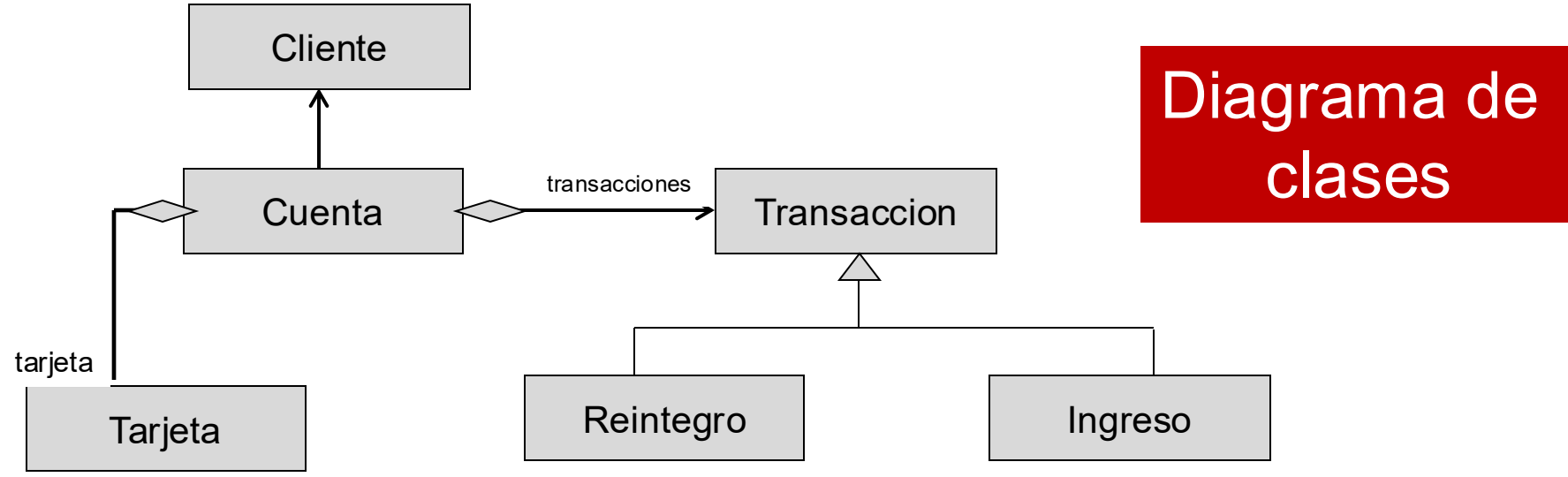
Clases vs Objetos (instancias)



Enlace

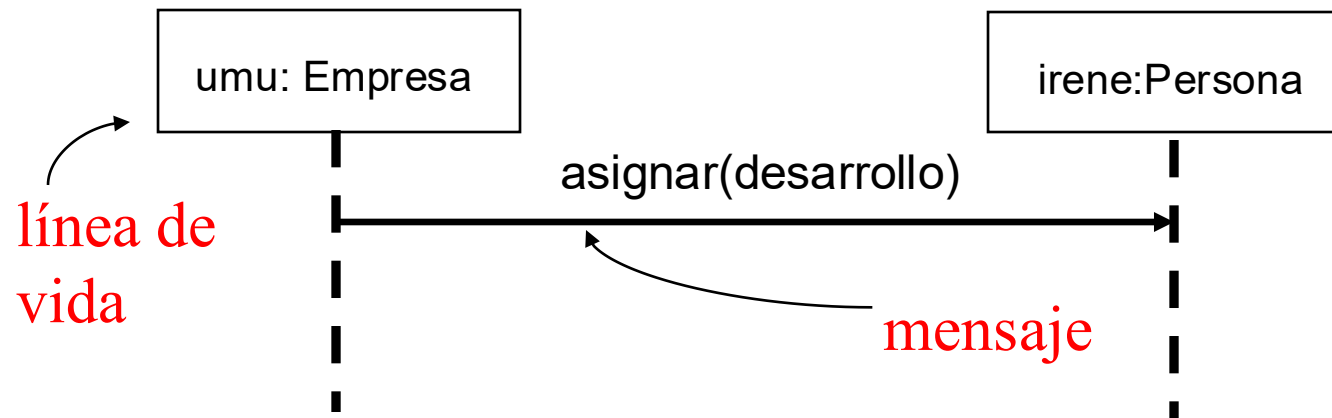
- Un **enlace** es una instancia de una asociación.
- Es un camino por el cual enviar un mensaje.



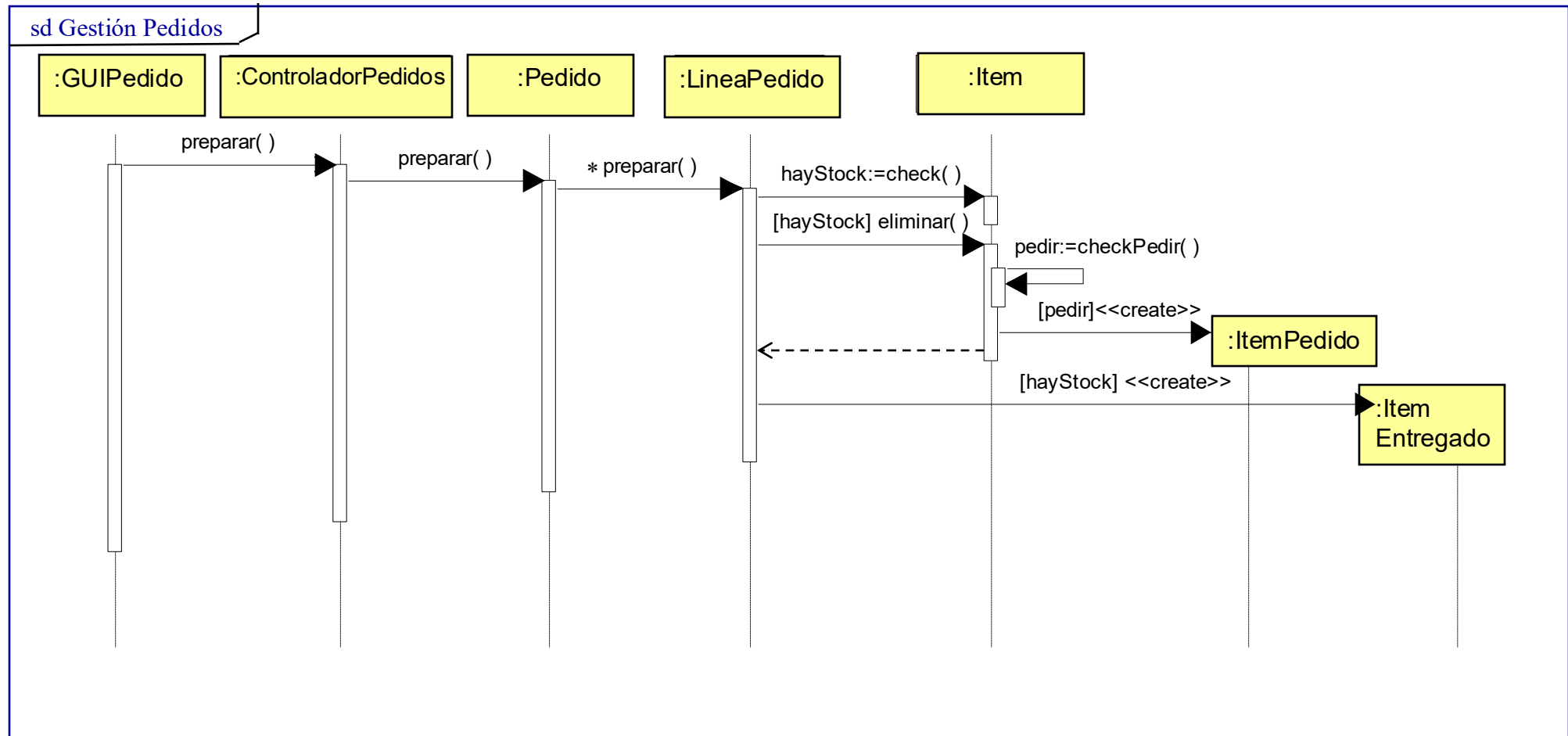


Diagramas de Secuencia

- Incluye:
 - **Líneas de vida**
 - **Focos de control o activación**
 - **Mensajes** a instancias o mensajes de creación
 - **Mensaje self**
 - Información de control: **condiciones** y **marcas de iteración**
 - Indicar el objeto devuelto por el mensaje: **return** (añadirlos sólo cuando ayuden a clarificar la interacción)

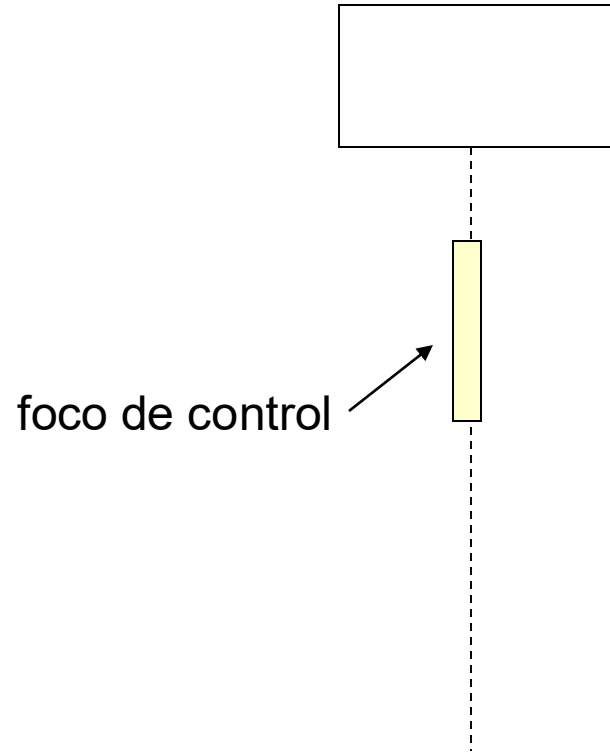


Ejemplo de Diagrama de Secuencia

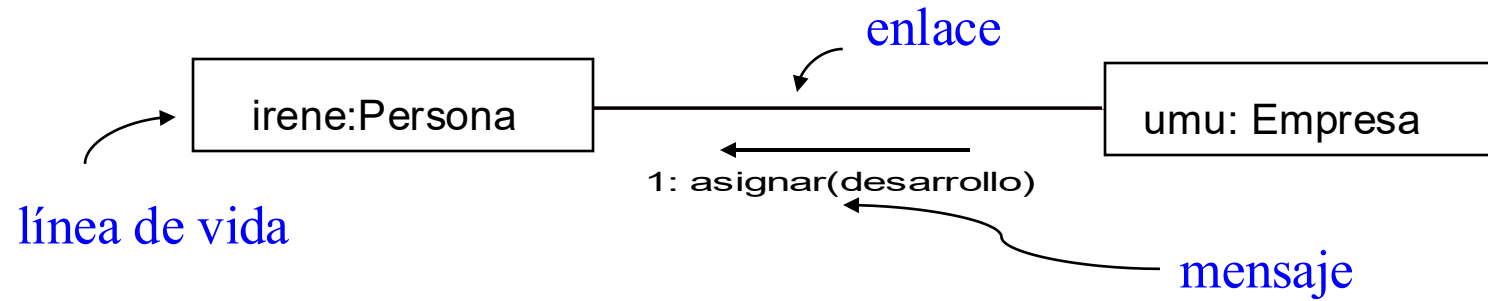


Foco de control o activación

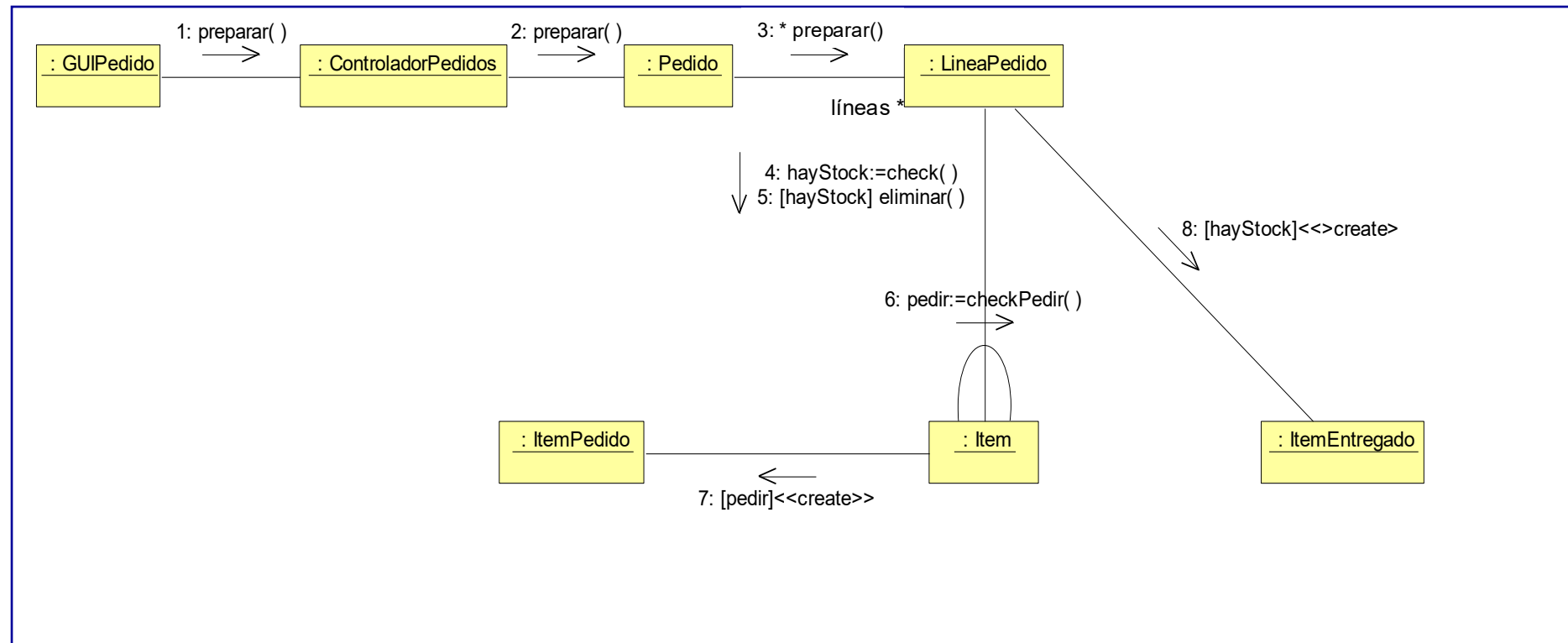
El **foco de control** es un rectángulo que representa el tiempo durante el que un objeto está activo ejecutando una acción.



Ejemplo de Diagrama de Colaboración o Comunicación



sd Gestión Pedidos



Ejemplo de Diagrama de Secuencia

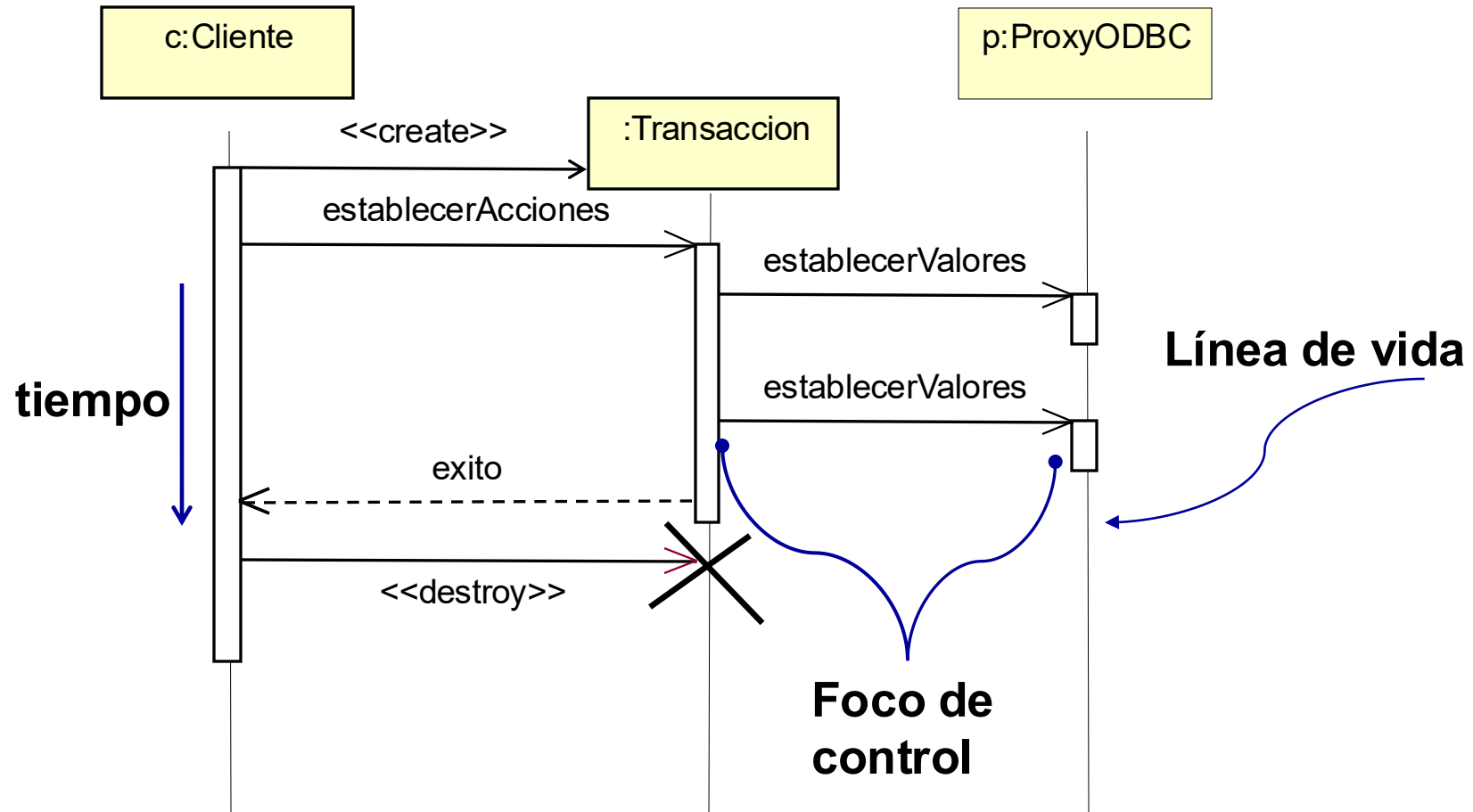
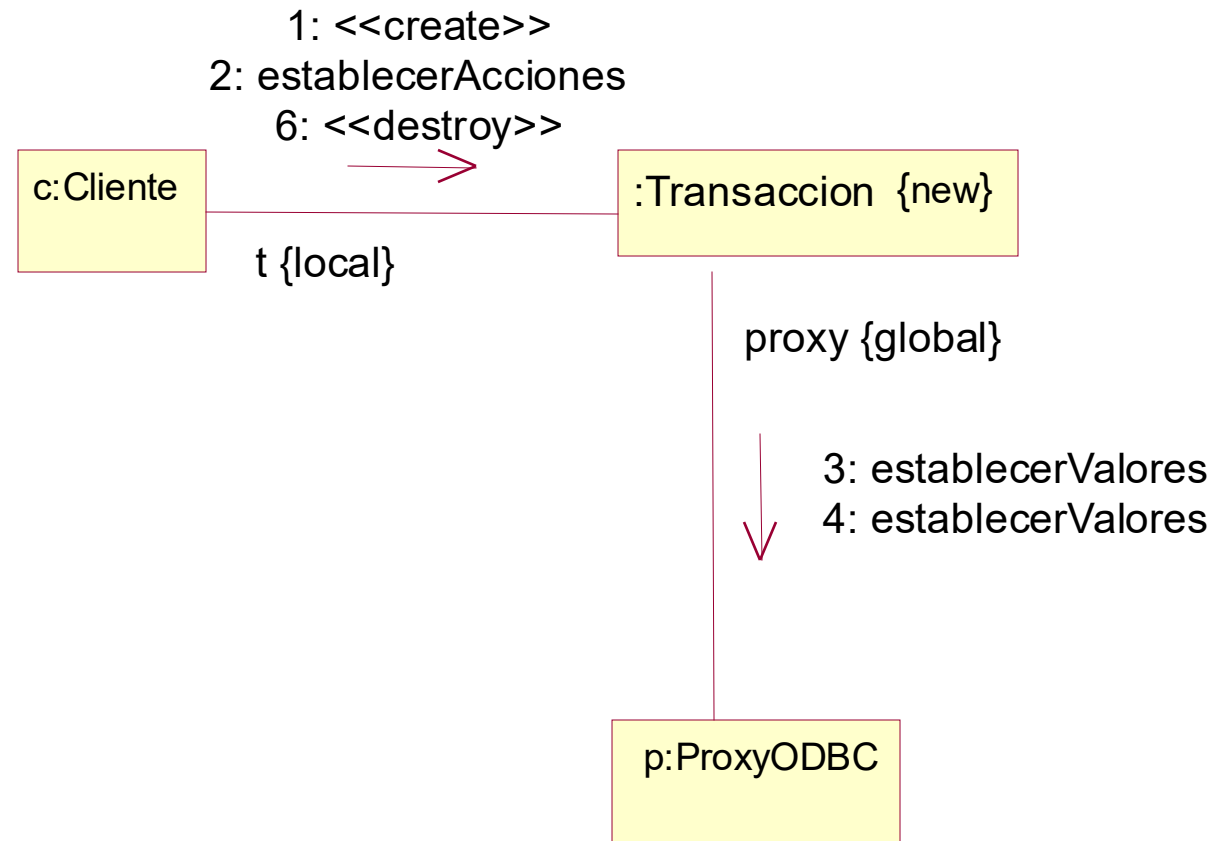
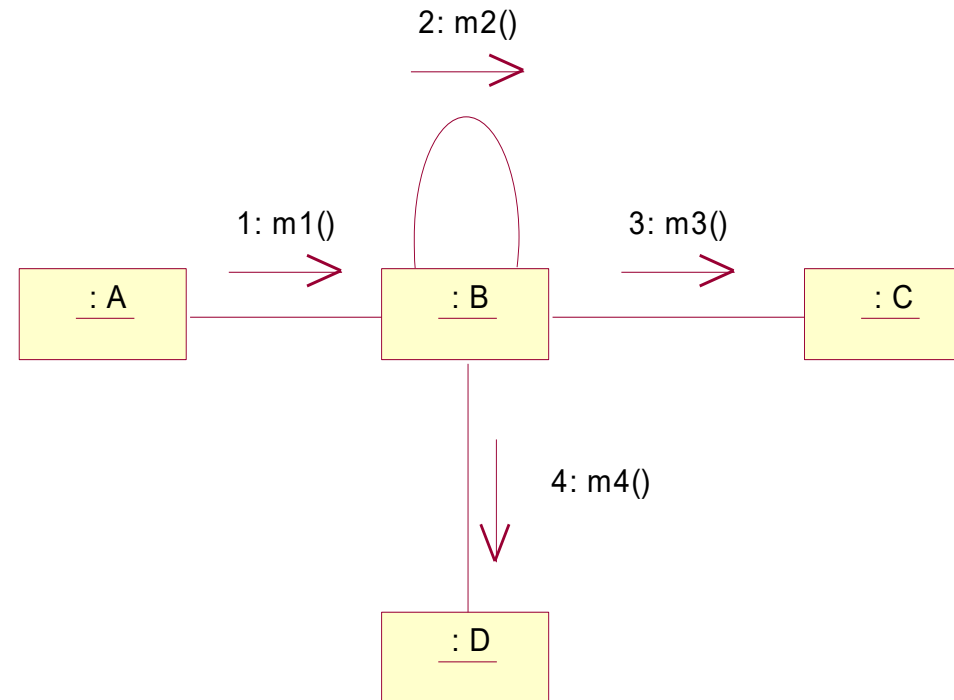


Diagrama de Colaboración. Equivalente al anterior

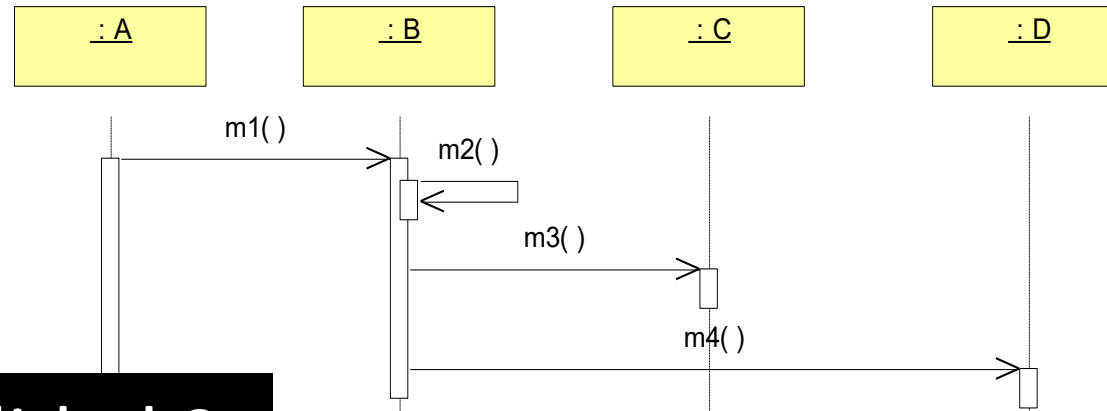


Numeración secuencial

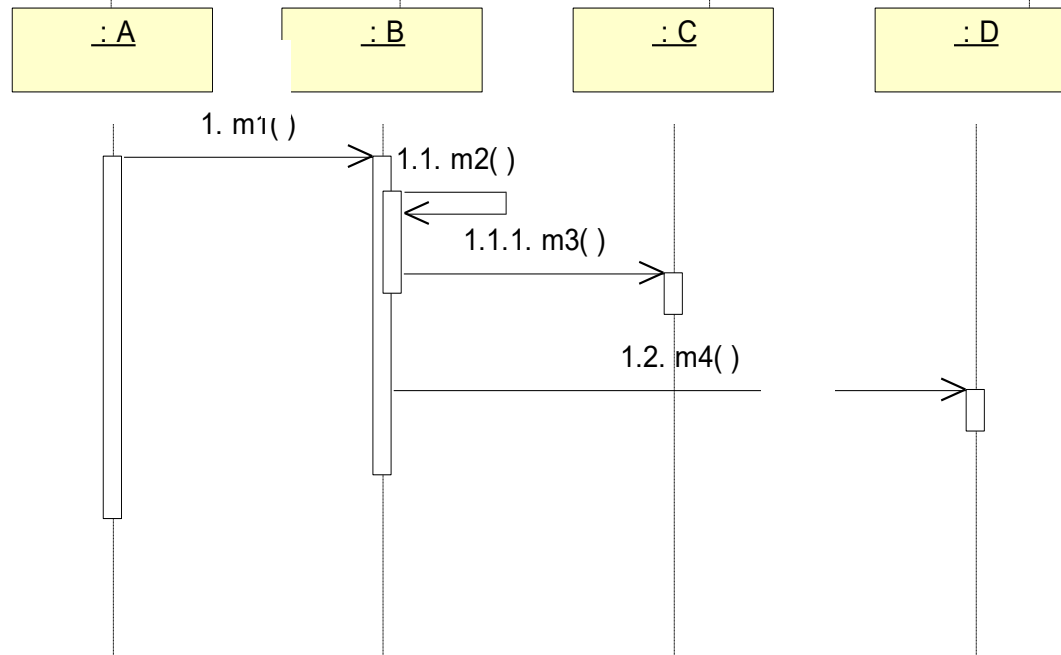


Confusión en el flujo de ejecución

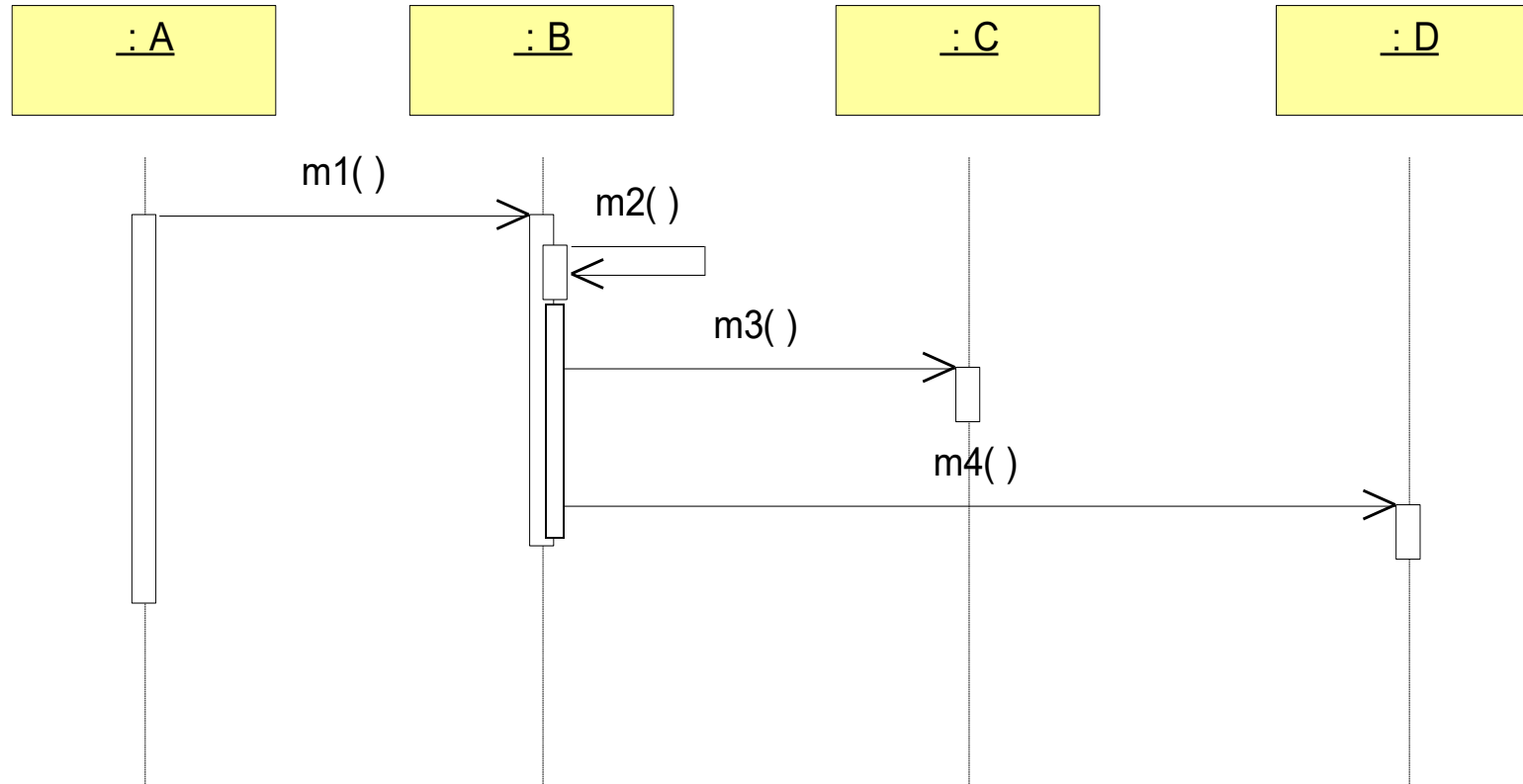
Posibilidad 1



Posibilidad 2



Posibilidad 3



Numeración jerárquica

